

任务图覆盖不到的级联损坏:无冲突柔性作业车间中的时空预约闭包

QI YU, Southeast University, PR China

WEI LI*, Southeast University, PR China

考虑运输的柔性作业车间调度若把运输时间取为按位置索引的常数矩阵,这一约定默认车辆互不共用同一时空资源,行驶时间因此只由布局决定,而不是路由决策或他车占用的结果。如果建模考虑了车辆对走廊的争用,有限车队在走廊上以时间窗预约通行,一辆车的让行会推迟另一辆车的到达。此时排程是否可行,便须同时满足任务依赖与预约表上的无冲突约束。在同时用工序依赖刻画加工先后、又用时间窗预约记录走廊占用的柔性作业车间调度中,扰动后重调度范围的现成做法仍是受影响工序重调度(AOR):先找出因设备故障或插单而无法按原计划执行的工序,再把同一工件的后续工序、以及同一机器上排在其后的工序一并纳入重排范围。这一做法覆盖了机器故障、插单等本身就是工序事件的扰动,却覆盖不到另一类——车间走廊在某段时窗内不可通行,或走廊上的车辆通行变慢。这类扰动不改任何工序指派:没有任何工序因此无法执行,AOR找不到起点,也就不重排任何工序;但原计划中落在阻断时段内的走廊占用已与禁行冲突,这些走廊占用规划不再可行,车辆阻塞或等待还会沿让行关系传给后续车辆。本文把这一缺口称作**任务图覆盖不到的级联损坏**:不是检测漏检,而是以工序为节点、以工件先后和同机工序先后为边的依赖图(任务图)不含走廊占用之间的让行关系,因而无法刻画走廊占用的规划失效如何在车辆之间传播。一旦模型把加工先后与走廊占用分成两套对象,却仍只用按工序是否受影响来划定重调度范围的方法,这种错位就会出现。

本文主张:当柔性作业车间用时间窗预约走廊来保证车辆无冲突通行时,扰动后的重调度范围必须定义在由走廊占用及其让行关系构成的时空预约图上,而不是定义在以工序为节点的任务图上——否则走廊阻断、通行变慢等不改工序指派的扰动会因任务图上没有受影响工序而被漏报,占用失效在车辆之间的传播也划不进重调度范围。为此我们提出时空预约影响闭包(STRC, Spatiotemporal Reservation Closure),即预约影响集:把占用时段与扰动时窗重叠的走廊预约作为起点,再沿四类依赖向外收录——同走廊上的让行、同一车辆的后续占用、同一工件下一道工序的占用、同一机器上下一道工序的占用;凡由起点沿这些关系直接或间接能够到达的预约,全部计入。影响集以内的占用允许改写,以外的沿用原计划。该集合是上述关系下的传递闭包:再走一步也不会走出集合。同走廊上谁先通过、谁让行,这一关系本身并不新鲜。本文不在排程时选择车辆的先后,而是在一份已经排好的计划上,按让行与接续关系,把会因起点占用失效而必须改写的后续走廊占用全部收录——包括被挡住的他车占用、同一车辆的后续路段、以及同一工件或同一机器上下一道工序对应的运输占用——得到由这些依赖关系唯一确定的影响集;而不是等重规划结束后,再根据实际改写了哪些走廊占用的时段或路径,事后倒推影响范围。随后在影响集内为原车重新规划路径,集外保持原占用不变;若改路后仍对不上后续工序或仍有冲突,再按同一工件的后续运输、同一车辆的后续占用、直至全部未完成占用的顺序,逐步扩大允许改写的范围。上述改路与扩大范围,只是把已经划定的影响集落到一份可行排程上;本文要主张的是影响范围应画在走廊占用及其依赖关系上,而不是提出一种新的修复策略。

实验取 5 个算例、每个算例 10 个随机种子,共 50 组对照。各方法共用同一套走廊路网、时间窗路由与无冲突校验,扰动时刻、阻断窗与计时口径也相同。对照包括:按工序依赖图划定重调度范围后再改路;以原排程为起点、在限定时间内做种群搜索的全局重解;以及两档不做种群搜索的全局重算——一档将未完成占用整体后推到阻断结束之后,一档保持原指派与加工顺序、为全部未完成运输重新规划路径。走廊阻断下,50 组中工序依赖图给出的影响范围均为空,预约影响集

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

均非空，且阻断后原排程均不可行。若改路程序相同、只改影响范围的计算方式，则按工序依赖图划界时 0/50 组恢复可行，按预约影响集划界时 50/50 组恢复可行。影响集在依赖边上向外封闭，即集内占用没有指向集外的依赖边；修复后集外每条走廊占用的走廊、时段与车辆均与原计划一致。这两项均在 50/50 组上成立。为单独检验预约影响集本身的作用，另设一对照：有界修复只改写影响集以内的走廊占用，影响集外沿用原计划；该对照则不再用影响集限制范围，把决策时刻尚未结束的走廊占用全部纳入改写。两者使用同一套改路程序，也都不得改写已经结束的走廊占用，差别仅在改写范围是影响集还是全部尚未结束的走廊占用。二者耗时与完工时间几乎不变（有界修复对它的完工时间为赢/平/输 4/46/0），但有界修复少改写约 7 个百分点的走廊占用（逐组 31/18/1， $p = 8.8 \times 10^{-7}$ ）。由此可以分开两项读数：毫秒级响应来自对影响集内各段运输各做一次路径规划、不重开种群搜索；预约影响集本身带来的是稳定性，即更少改写原计划中的走廊占用。有界修复耗时的中位数为 3.2ms。同一套实现中，保持原指派与顺序、为全部未完成运输重新规划路径的那一次全局重算，耗时的中位数仍高于一次有界修复；唯一更快的是全局右移：不判断哪些占用受影响，把未完成占用整体后推到阻断结束之后，但其完工时间更差，改写的占用也更多。相对不改写已完成占用的热启动全局重解，有界修复的耗时仍低两至三个数量级；在全部 18 个时间预算点上，有界修复的完工时间均较差，不过差距已明显小于从零时刻起全部重排。两边被改写的占用比例同在 50%–66% 量级——这一接近不能记成影响集带来的稳定性，必须与「只差允许改写哪些占用」的对照一起读。将算例从 8 工件 4 台加工机器推至 20 工件 10 台加工机器，影响集占全部占用的比例不升反略降。在全部 18 个时间预算点上，有界修复的完工时间均劣于热启动的全局重调度，不存在随预算放宽而反超的交叉点。完工时间上，有界修复因此不能替代全局重调度。走廊阻断发生后，若只按工序依赖图判断，重调度不会启动，原排程保持不可行；若只标出损伤而不改写占用，冲突同样还在。全局重调度可以在观测到阻断后随时启动，但耗时在秒级以上。当必须在毫秒内给出可执行排程时，需要的是与时间窗预约相匹配的局部恢复，而不是再开一次搜索。影响集也不是恢复可行所必需的最小占用集合：它覆盖决策时刻尚未结束的走廊占用约 92%；稳定性收益的幅度与它未纳入、因而保持不动的占用多少成正比，故在影响集接近「全部尚未结束的占用」的算例上，这一收益近乎消失。本文的贡献是把扰动后的重调度范围从工序依赖改画到走廊占用及其让行、接续关系上，并给出与该划界相匹配的局部恢复步骤。

CCS Concepts: • **Theory of computation** → **Scheduling algorithms**; • **Applied computing** → **Industry and manufacturing**; • **Computing methodologies** → **Planning and scheduling**.

Additional Key Words and Phrases: 柔性作业车间调度, AGV 无冲突路由, 时空预约表, 影响闭包, 走廊阻断

ACM Reference Format:

Qi Yu and Wei Li. 2026. 任务图覆盖不到的级联损坏:无冲突柔性作业车间中的时空预约闭包. 1, 1 (September 2026), 30 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 引言

1.1 背景与动机

考虑运输的柔性作业车间调度长期建立在一个便利假设之上：工件在两台机器之间的移动时间是一个常数，可从按位置对索引的矩阵中读出。该假设默认车辆互不共用同一路段，行驶时间因此只由两点之间的距离决定，而不是路由决策的结果。一旦有限车队共享稀疏走廊，并以时间窗预约被争用的路段，行驶时间就不再是固定数据，而成为两个决策的结果——车辆走哪条路，以及谁先占用了该路段。一辆车的让行会推迟另一辆车的到达，进而推迟后续工序开工及其运输占用。此时一份可行排程须同时确定工序指派、加工顺序，以及一张记录走廊占用的预约表；扰动不仅可以打坏工序，也可以只使走廊占用失效，而不改任何工序指派。

在开工前一次性完成排程、并把机器指派与无冲突路径一同决定的这类研究已经表明：当工序可选多台机器时，车辆在走廊上的争用会改变哪台机器更优——通往加工更快的机器的路段若正被占用，加工较慢但更易到达的机器反而更好。因此，评价候选排程须用时间窗路由计入等待，搜索时还须查询预约表上的冲突，而不能继续使用常数行驶时间 [9, 17, 20]。上述文献回答的是：在信息完备、尚未开工时，如何排出一份高质量的无冲突排程。同一车间在执行到时刻 t_{now} 并发生扰动之后，问题改为：损坏落在哪些走廊占用上、改写到何处才足以恢复可行，以及这件事能否在不重新开种群搜索的前提下，在毫秒至百毫秒内完成。

1.2 任务图覆盖不到的缺口

在考虑运输的柔性作业车间动态调度研究中，重调度范围通常沿工序依赖图划定：以因设备故障而无法按原计划执行的工序为起点，再把同一工件的后续工序、同一机器上排在其后的工序纳入范围，或在依赖图上从起点起限定步数 θ 后停止，然后只对范围内的工序重调度。两篇带 AGV 运输、并以机器故障或插单为扰动的动态调度研究即按此划界——恢复范围分别取「故障后尚未加工的工序」与「按扰动传播分类后的工序与配送任务」。这一做法可上溯至受影响工序重调度（AOR）[19, 24]。后文设一道按工序依赖图划界的对照，用以复现这一现仍沿用的做法，而不是改进 AOR 本身（第 2.1 节）。

这一划界在扰动本身就是工序事件时往往合理——例如机器故障使若干工序无法按原指派执行。然而，当排程用时间窗记录走廊占用时，还存在一类扰动：它使某条走廊在某段时间内不可通行，而不改任何工序指派。典型例子是走廊阻断：走廊在 $[t_a, t_b)$ 时段禁止通行。阻断不修改机器指派或加工顺序，工序依赖图上因此没有任何被直接波及的工序，按该方法划出的重调度范围为空，并据此判为无需重调度。于是工序依赖图覆盖不到这些必须恢复的走廊占用：原计划中与阻断时段重叠的占用已与禁行冲突，不再可行；一车因此等待或改道，又会沿让行关系挡住后续车辆，失效在预约表上传播。

这里的「覆盖不到」，指工序依赖图按定义不含走廊占用之间的让行关系，因而无法刻画占用失效如何在车辆之间传播，而不是传感器漏检或算法偶然没查到。若运输时间仍取为按位置对索引的常数矩阵，走廊不是可争用的资源，预约表也不存在，这类损坏甚至无法写进模型。因此，该缺口只在用时间窗记录走廊占用的设定下能够成立；这不是把应用范围故意收窄，而是把问题写清楚所必需的前提。

据此，我们区分两类扰动。

- **A 类**（作用于工序依赖图）：机械臂故障、加工时间显著偏差、插单，以及车辆故障。这类扰动使部分工序无法按原计划执行，划定重调度范围时的起点工序集非空；按工序依赖图划界与按预约影响集划界都能给出非空范围，因而不是本文用来检验划界错位的主要对象。车辆故障归入 A 类的理由见第 3.3 节：它虽不改工序指派，却使该车承运的工序不可执行，经「车辆到所承运工序」的对应关系，工序依赖图上仍有非空的起点工序集。
- **B 类**（只作用于预约表）：走廊阻断、走廊通行变慢，以及局部通行权临时取消等。这类扰动不使任何工序无法执行，工序依赖图上的起点工序集为空；与受扰时段重叠的走廊占用非空，由其收录得到的预约影响集也非空。

B 类构成问题层面上的反例：问题不在于现有方法对工序的重调度修得不够好，而在于同时用工序依赖图刻画加工先后、又用时间窗记录走廊占用的模型里，现成的重调度范围仍划在工序依赖图上，而该图表达不了走廊阻断这类扰动。这一错位的条件是模型里同时有这两套对象，却只用工序那一套划界。上引两篇带 AGV 的动态调度研究没有出现「范围画在工序图上、因而漏掉走廊阻断」这一问题，是因为它们在模型中不考虑车辆之间的走廊争用（第 2.4 节），于是只剩下工序依赖图可用来划定重调度范围——这说明错位是在两套对象同时出现时才会发生，而不是某一篇工作的疏忽。因此，本文的主张只针对上述设定，而不是认为所有局部重调度文献都存在这一错位（第 2.5 节）。图 1 对照这一缺口：左侧工序依赖图上没有作为起点的工序，因而判为无需重调度；右侧预约表上，与阻断时段重叠的占用成为起点，并沿让行与接续关系形成预约影响集。

1.3 本文主张

本文有三点主张。

第一，当柔性作业车间用时间窗预约保证车辆无冲突通行时，扰动后的重调度范围应定义在由走廊占用及其让行关系构成的时空预约图上，而不是只定义在工序依赖图上。

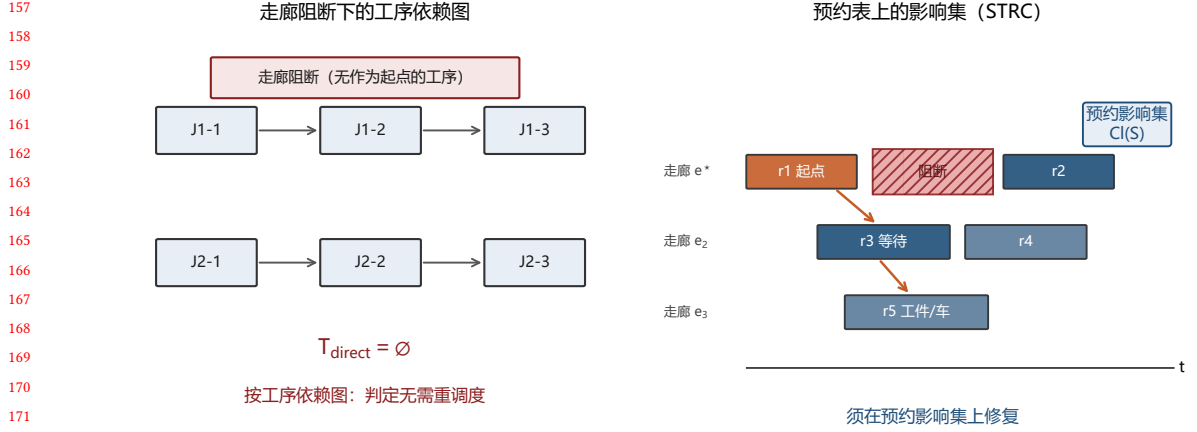


Fig. 1. 动机示意。左:走廊阻断不使任何工序或机器无法按原计划执行,工序依赖图上没有作为起点的工序。右:同一扰动下,与阻断时段重叠的走廊占用成为起点,并沿让行与接续关系形成预约影响集。

第二,划界对象是时空预约影响闭包 (STRC, Spatiotemporal Reservation Closure),即预约影响集:以与扰动时窗重叠、因而失效的走廊占用为起点,沿让行、同一车辆的后续占用、同一工件下一道工序的占用、同一机器上下一道工序的占用,把直接或间接必须作废、重写的走廊占用全部收录。这些依赖关系一经给定,影响集以外的占用不再是集内占用沿上述关系的下一步,因而「改写哪些占用」对应一个可以检验的集合,而不是人为设定的搜索半径。

第三,得到影响集之后,在集内为原车改路、集外保持原占用;若仍得不到可行排程,再按同一工件、同一车辆、直至全部未完成占用的顺序,扩大允许改写的范围。A类故障上另允许换车与改派,仍不改加工顺序。这些步骤用来落实影响集并恢复可行排程;本文的主张在于如何划定影响集,而不在这些修复步骤是否新颖。

上述三点共用一个立场:预约影响集是一次测量,不是一个决策。本文不用「谁挡了谁」去引导搜索以更换指派或路径——那是在改进排程决策;消去一处走廊等待之后,另一处约束往往会成为新的瓶颈,完工时间上的收益因此容易被抵消。本文用同一份「谁挡了谁」去划定哪些占用已经损坏:依赖关系给定后,一条占用是否属于影响集,取决于它能否从起点沿这些关系到达,结果唯一确定,不随之后哪一条约束成为新的瓶颈而改变。

这并不等于声称影响集已经包含物理上所有必须改写才能恢复可行的占用。画哪些依赖边、不画哪些,是建模时的选择;边集一旦选定,影响集对外封闭(集内占用没有指向集外的依赖边)就是该定义的推论(命题2),不是另外证明出来的性质。影响集是否为恢复可行所需的最小占用集合,本文不作主张:最小集合依赖于允许哪些修复动作,而影响集刻意不绑死某一种修复方法(注2)。

相对以原排程为起点、在限定时间内重新搜索的全局重调度,预约影响集回答的是另一类问题:在必须很快给出可执行方案时,能否恢复可行,并把改写限制在影响集以内。修复后,影响集以外每条走廊占用的走廊、时段与车辆均与原计划一致——这是边集给定、且集外占用已按原计划成功写入新表时的充分条件(命题3),不是无条件成立的等式。相对不改写决策时刻已经结束的全局重调度,有界修复得到的完工时间 C_{max} 更差,两边被改写的占用比例已在同一量级;其长处在于算法耗时短得多(毫秒对秒)。算法耗时与改写比例不应混在一处解释,第6.9节已分开测试:耗时短来自对影响集内各段运输各做一次路径规划、不重

开种群搜索，并不来自影响集本身；至于影响集是否少改写了占用，不能从上述全局重调度对照中读出——两边比例已经接近——而须另设一对照：改路程序相同，唯一差别是允许改写的是影响集，还是尚未结束的全部占用。在本文测到的每一个搜索时限上，全局重调度得到的 $C_{\max}$ 都更小，有界修复没有在完工时间上反过来更优。因此，有界修复的作用不是降低完工时间，而是在毫秒内给出可执行的修复方案。按工序依赖图划界时，走廊阻断因找不到受影响工序而不会启动局部重调度，原排程保持不可行。同一扰动若改为启动全局重调度，则可以处理，但算法耗时在秒级。当必须立刻恢复可行时，需要先识别走廊占用上的损坏范围，再选用与之匹配的局部恢复步骤。本文解决的不是找到一种完工时间上更优的重调度算法，而是这件识别与恢复的事。

1.4 贡献

本文的贡献如下。

- (1) **指认工序依赖图覆盖不到的一类损坏。** 当排程同时用工序依赖刻画加工先后、又用时间窗记录走廊占用时，现成的重调度范围仍按受影响工序划定。走廊阻断、走廊通行变慢等扰动不改任何工序指派，按该方法划出的范围为空，并据此判为无需重调度；但与受扰时段重叠的走廊占用已经失效，原排程不可行，等待还会沿让行关系传给后续车辆。这一缺口不是检测漏检，而是工序依赖图不含占用之间的让行关系；运输时间若仍取为常数矩阵，走廊不是可争用资源，这类损坏甚至写不进模型。
- (2) **把重调度范围定义为预约影响集。** 以与扰动时窗重叠的走廊占用为起点，沿让行、同一车辆的后续占用、同一工件与同一机器上下一道工序的占用，收录直接或间接必须作废、重写的记录，得到时空预约影响闭包（STRC）。依赖关系一经给定，一条占用是否属于影响集唯一确定，集内占用没有指向集外的依赖边，因而「改写哪些」对应一个可检验的集合，而不是人为设定的搜索半径，也不随某一种改路算法如何实现而变。影响集是一次测量，不是一个排程决策：本文用「谁挡了谁」划定损坏，而不用它去引导搜索更换指派或路径。本文不主张影响集已是恢复可行所需的最小占用集合（最小集合依赖于允许哪些修复动作，注 2）。
- (3) **给出与该划界相匹配的局部恢复，并分清各项读数各来自何处。** 在影响集内为原车改路、集外保持原占用；仍不可行时再扩大允许改写的范围。这些步骤用来落实影响集，本身不是本文的方法贡献。在同一套走廊路网、时间窗路由与无冲突校验下对照表明：按工序依赖图划界时走廊阻断 0/50 恢复可行，按预约影响集划界时 50/50 恢复可行，可行与否来自划界对象，而不是改路程序的优劣；有界修复以毫秒级耗时恢复可行，但完工时间 $C_{\max}$ 在全部搜索时限上均劣于全局重调度，不存在交叉点；相对「改写全部尚未结束的占用」的对照，耗时与 $C_{\max}$ 几乎不变，少改写的占用才可归因于影响集（第 6.9 节）。四类扰动与两种划界的交叉中，工序图范围仅在走廊阻断与通行变慢上为空，预约影响集均非空，且在机器或车辆故障上不小于工序图范围。

同一车间、同一预约表上，开工前的一体化排程用预约表构建无冲突方案 [9, 17, 20]；本文在执行时刻用预约表定位损坏范围。二者时机不同，并不互相替代。

1.5 篇章结构

第 2 节按四类回顾相关工作：作业车间如何按工序或时间轴划定重调度范围、铁路调度如何表达排他资源上的占用先后、无冲突多车路路径如何做局部重规划，以及考虑运输的柔性作业车间把边界划在何处，并据此说明本文的位置。第 3 节给出走廊预约设定与执行期恢复问题：车间、走廊预约表与工序依赖图，两类扰动，以及恢复须满足的条件。第 4 节定义时空预约影响闭包，即预约影响集，说明其依赖边与对外封闭等性质（图 2、3）。第 5 节说明如何在影响集内为原车改路、集外保持原占用，以及仍不可行时如何扩大允许改写的范围，

并与全局重调度对照（图 4）。第 6 节报告可行性、算法耗时、占用改写比例与完工时间，并用案例甘特把统计结果落到工序时间轴上。第 7 节总结全文，并讨论局限与后续工作。

2 相关工作

动态调度的一个基本问题是：扰动发生后，重调度范围划在什么对象上，即改哪些、其余保持不动。已有文献把这一范围划在不同对象上——工序或时间轴、排他资源上的占用先后、受影响的车辆，以及带运输的柔性作业车间中的工序集合。下面按这四类回顾各自做了什么、尚未覆盖什么，并在表 1 中做汇总。

2.1 作业车间的局部重调度

对机器故障、插单等工序事件，按受影响工序划定重调度范围，在带 AGV 的动态调度中仍被采用。Zhang 等 [24] 处理机器故障与插单，恢复范围取故障后尚未加工的工序；Tang 等 [19] 按扰动传播把工序与配送任务分为保留、继续、重构，并把完全重调度、部分重调度与右移并列，指出右移保住稳定性、让出优化空间。这一做法可上溯至受影响工序重调度(AOR) [1, 16]：只重排被扰动直接或间接波及的工序。沿时间轴划界与失败后扩大范围同样已有文献研究过，包括 match-up [2] 与增量扩大时间窗的 Local Rescheduling [8]。右移则不判定谁受影响，把未完成部分整体后推。滚动时域与分解式方法在规模与质量之间折中 [23, 26]；综述见 [14, 21]。

这些工作表明，划定范围再局部重调度并不是新问题，前提是扰动首先作用在工序上。一旦扰动只使走廊不可通行而不改指派，按该方法划出的范围可以为空，原排程却已不可行。

2.2 排他资源上的占用先后

「谁先占用同一排他资源」被建成图，在铁路调度中已有系统研究。Mascis 与 Pacciarelli [12] 的 alternative graph 把析取图推广为：顶点是作业对闭塞区段的占用，固定弧表示路径先后，成对的 alternative arc 表示谁先通过该区段。D'Ariano 等人 [4] 用该模型做列车实时重调度，扰动后重算无冲突时刻表，并尽量接近原计划。同走廊上的让行关系与 alternative arc 表达的是同一类先后。

与本文不同的是用法。对每一处争用，alternative graph 给出两条互斥的弧（甲先通过，或乙先通过），排程就是从中选定一条：选定后图中不能出现正权回路，时刻表才可行，并尽量使目标最优——占用先后因此是决策变量。本文的计划已经排好，谁先通过哪条走廊已经确定，不再选择先后，只按这些既定的让行与接续关系，把扰动后必须作废的原走廊占用纳入重调度范围。此外，铁路模型把闭塞区段当作机器，工序与占用在同一张图上，区段不可用就是工序事件，不存在「范围画在工序图上、因而漏掉走廊阻断」。柔性作业车间里加工与走廊占用是两套对象，现成的 AOR 划界用不上这一套占用先后。

2.3 无冲突多车路径的局部重规划

在多智能体路径规划中，只为受影响者进行重调度、其余视为动态障碍，已是标准做法。Švancara 等人 [18] 在线 MAPF 中给出 Replan All、Replan Single 与 Replan Affected，并用在线独立性检测确定受影响集合；Malka 等人 [11] 在地图不准确时，同样只为受观测变化影响的车辆重规划。无冲突路径规划把时空占用作为一等约束，但起终点通常由外部给定 [25]。

上述做法已经把「只改受影响者、其余保持原计划」做成标准机制，本文并不把它当作贡献。与本文设定的另一点不同是：这类工作只有车辆的起终点，没有工件—工序—机器指派，因而不会出现「重调度范围按工序依赖图划定，走廊阻断因找不到受影响工序而被漏掉」。就受影响集合如何得到而言，MAPF 用独立性检测找出受影响车辆，得到的是检测算法的输出，随实现方式和停止条件而变。本文给出的预约影响集则不同：依赖边一经给定，一条占用是否属于该集，只取决于它能否从起点沿这些边到达，与随后用哪一种改路算法无关（第 4.2 节）；对外封闭是这一定义的推论，规模即该集合的大小。

表 1. 相关工作按划界对象分类。

类别	代表工作	边界划在	有工序依赖图?	与本文的关系
作业车间局部重调度	AOR [1, 16]; match-up [2]; LRS [8]; 带 AGV 的动态调度 [19, 24]	工序 / 时间窗	有	后文对照复现按工序划界与右移
排他资源占用先后	alternative graph [12]; 铁路实时重调度 [4]	闭塞区段占用	有,与占用在同一张图	让行关系同类; 铁路用它选择车辆的先后, 本文用既定先后划定需要改写的占用
无冲突多车路径	Online MAPF [11, 18]; LMAPF [25]	受影响车辆	无	集内改路、集外保持已是标准做法
带运输的柔性作业车间	常数矩阵 [5, 13, 15, 22]; 一体化排程 [9, 17, 20]	工序集合	有;预约表与执行期扰动未同时出现	本文的位置

2.4 考虑运输的柔性作业车间调度

考虑运输车辆的柔性作业车间调度已有系统综述 [22]。近期组合优化工作仍普遍把行驶时间取为按位置对索引的常数矩阵, 并在此之上发展约束规划、元启发式与知识引导搜索 [5, 13, 15]。该约定下车辆并不争用同一路段, 走廊阻断在模型里没有对应对象, 无法被检验。

一旦用时间窗记录走廊占用, 加工先后与走廊占用同时进入模型。拥堵感知派车可用局部密度等代理量选车, 而不改机器指派 [7]。把柔性指派与无冲突运输真正合在一起的工作相对较少: van Os 与 Basten [20] 形式化了带无冲突运输约束的柔性作业车间调度, 并以分解方法给出可证最优基准; 两阶段方案先固定排产再消解车辆冲突 [17]; 亦有学习层排产、路径层消冲突、并以有限反馈连接的框架 [9]。但这些工作回答的是开工前如何排出可行且质量好的无冲突排程。

把走廊预约与执行期扰动同时纳入的工作目前还没有。模型中没有预约表、即使引入了预约表也把执行期故障排除在外: Zhang 等 [24] 并写明忽略 AGV 之间的碰撞; Sun 等 [17] 用时间窗 Dijkstra 消解车辆冲突, 却假定车辆始终可用、不考虑故障与充电, 并把设备维护与故障列为后续研究。

因此, 缺口不是某篇已有文献把边界画错了, 而是两个条件至今没有被融合: 工序级划界成立, 往往因为没有走廊预约; 已有预约表的设定, 又把执行期扰动排除在外。一旦两者同时出现, 现成可沿用的只有接受影响工序划界的做法, 它对不改工序指派的走廊阻断会给出空范围(第 6.2 节实测 50/50)。本文要处理的就是这一情形。

2.5 本文定位

综上, 在考虑 AGV 无冲突路由的柔性作业车间中, 排程同时用工序依赖刻画加工先后, 又用时间窗记录走廊占用。此时会出现一类已有文献尚未处理的情形: 走廊阻断等扰动并不使工序无法执行, 原指派仍然有效, 但与受扰时段重叠的走廊占用已经失效, 原排程不可行。按受影响工序划界会得到空范围, 重调度不会启动; 持有预约表的一体化排程工作则通常不考虑执行期扰动。本文研究这一问题, 是因为现成划界覆盖不到这类损坏, 需要先识别哪些占用必须改写。后文按工序依赖图划界的对照, 复现的是第 2.1 节的做法。本文主张把重调度范围改到走廊占用及其让行、接续关系上, 得到由这些关系唯一确定的预约影响集。

3 走廊预约设定与执行期恢复问题

3.1 系统设定与记号

问题设定沿用带无冲突运输约束的柔性作业车间 [20]。本节一次写清后文用到的对象。一份可行排程 S^0 视为给定输入: 它在 $t = 0$ 如何被搜索出来, 不影响后文如何划定须改写的占用。

3.1.1 车间与四组决策. 车间路由图 $G = (V, E)$ 强连通。机械臂占据固定节点 $\text{loc}(m)$; $v_0 \in V$ 为装卸站。 n 个工件、机器集合 M 、同构单载车队 $K = \{1, \dots, N_A\}$ 。工件 j 有固定工序链 $O_{j1} \rightarrow \dots \rightarrow O_{jn_j}$; 工序 O_{ji} 的候选机器集为 $\Omega_{ji} \subseteq M$, 在机器 m 上耗时 t_{jim}^P 。一份完整排程同时确定四组决策: (i) 每道工序的机器指派 x ; (ii) 各机器上的工序扫描序 π ; (iii) 各次搬运的车辆指派 y ; (iv) 每次车辆移动在 G 上的无冲突路径。记

$$S = (x, \pi, y, R),$$

其中预约表 R 是第 (iv) 组的显式记录, 定义见下。

运输任务由指派派生, 不是输入: 仅当相邻两道工序不在同一节点时才产生一次搬运。一次搬运拆成两段——空载取货与满载送达——各段有自己的任务标签 u (例如 O_{ji} 的空载段与满载段是两条记录)。后文允许改写哪些占用、哪些占用保持原计划, 以及如何改路, 都以段为粒度, 而不是整道工序。

3.1.2 走廊预约与无冲突. 每条走廊 $e \in E$ 是排他资源, 通行时间 τ_e 已知; 两个方向共享同一份占用, 因而任一时刻至多一辆车占用 e , 迎面冲突与同向超车一并排除。占用取半开区间: 第二辆车可以恰好在区间右端点进入。节点上设容量充裕的停靠位, 等待发生在节点上且总是被允许。一条预约记录为

$$r = (e, [t^s, t^e], k, u) \in R,$$

即车辆 k 在 $[t^s, t^e]$ 占用走廊 e , 所属运输段为 u 。同一走廊上任意两次不同占用必须满足 $[t^s, t^e] \cap [t^{s'}, t^{e'}] = \emptyset$ 。路径只写入 R 报告为空闲的时窗, 故无冲突是构造出来的, 不是事后修补的。

时间窗路由是后文改路与各对照方法共用的原语: 给定起终点、最早出发时刻与当前 R , 它返回一条与已有占用无交的通路及各走廊进入时刻。实现上采用时间窗 Dijkstra [17]; 本文不改这条原语, 只改哪些运输段被送进路由器。与 van Os 预约节点、Lyu 另加节点容量不同 [10, 20], 本文预约的是边; 第 7.1 节交代由此带来的算例口径限制。

3.1.3 工序依赖图与预约依赖图. 工序依赖图 $G_T = (T_T, E_T)$ (任务图) 以工序为顶点, 边为工件先后与同机先后; 它不含走廊占用之间的让行关系。预约依赖图 $G_R = (R, E_R)$ 以 R 中的占用为顶点, 边为让行、同车、同工件与同机所诱导的依赖 (定义 4)。两张图的顶点集不相交: 一边是加工, 一边是通行。因此存在一类只改走廊时窗、不改任何工序参数的扰动——按 G_T 划界时找不到作为起点的工序, 而按 G_R 划界时起点占用非空, 且原排程已不可行。图 2 对照这两张图; 图 1 给出走廊阻断下的后果。

3.2 执行期假设

执行期约定与物理设定如下。对照实验共用这些约定, 以便把读数归因到重调度范围如何划定, 而不是模型差异:

A1. 初始排程可行. S^0 在扰动前通过无冲突校验; 预约表与工序时间表一致。

A2. 已完成占用不回溯. 只改写 $t_{\text{end}} > t_{\text{now}}$ 的未来占用与未完工工序; $t_{\text{end}} \leq t_{\text{now}}$ 的占用字段不可改写。

A3. 单次、完全观测. 扰动在 t_{now} 被完整观测, 且不与其它扰动叠加; 扰动流、预测误差与滚动响应不在本文范围内。

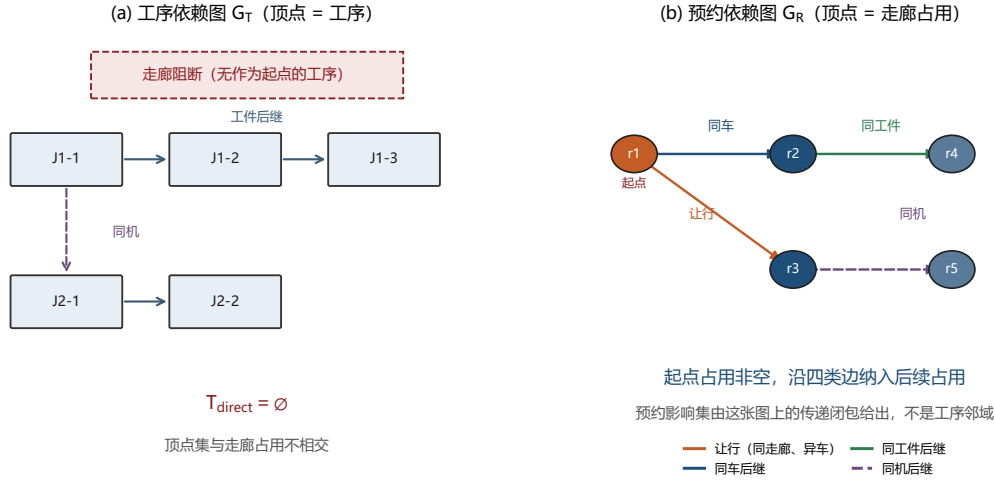


Fig. 2. 两张图的顶点集不相交。(a) 工序依赖图 G_T , 顶点是工序, 走廊阻断找不到作为起点的工序; (b) 预约依赖图 G_R , 顶点是走廊占用, 从与阻断窗重叠的占用出发, 沿四类依赖边纳入后续占用。

表 2. 本文常用记号

符号	含义
$G = (V, E), \tau_e$	走廊路网; 走廊 e 的通行时间
$v_0, \text{loc}(m)$	装卸站; 机械臂 m 所在节点
x, π, y	机器指派、扫描序、车辆指派
$R, r = (e, [t^s, t^e], k, u)$	预约表; 一条半开占用 (车辆 k , 段标签 u)
G_T / G_R	工序依赖图 / 预约依赖图
S^0, S'	扰动前排程、恢复后排程
$\mathcal{D}, t_{\text{now}}$	扰动事件、发生 (决策) 时刻
$T_{\text{direct}}, T_{\text{impact}}$	工序依赖图上直接受扰的工序 / θ -步纳入范围
$\text{Seeds}(\mathcal{D}), U$	作为起点的占用; 允许改写的占用集合
$\text{CI}(\cdot)$	预约影响集 (STRC)
φ	受扰未来工序比例 (规模扫描)

A4. 走廊排他、缓冲充裕、车队同构单载。 走廊仍为排他半开资源, 阻断以附加占用 (或禁行窗) 注入路由层。有限缓冲、充电、加减速不建模; 全部车辆在计划开始时空载停于 v_0 。

A5. 允许的恢复动作。 B 类扰动保持原机器指派 x 与扫描序 π , 只改路; 失败时扩大允许改写的占用集合。A 类故障额外允许: 车辆故障把故障车从车队摘除并换车; 机械臂故障把未完工序改派到其它可行机。扫描序 π 始终不变, 不重开搜索。实验中的全局重调度对照可以改 x 与 π (第 5.3 节)。

3.3 扰动二分

定义 1 (A/B 类扰动). 若扰动直接使某些工序的指派或可执行性失效 (机械臂故障、显著工时偏差、插单、车辆故障等), 称其为 **A 类**; 若扰动只使某些走廊时窗不可用、而不使任何工序不可执行 (走廊阻断、走廊通行变慢等), 称其为 **B 类**。

注 1 (车辆故障为何属于 A 类). 车辆故障不改动任何工序的机器指派,乍看应与走廊阻断同类。但定义 1 的判断是可执行性而非指派:一辆车抛锚后,它承运的那些工序在原计划下无法开工,故经「车辆→所承运工序」的映射,工序依赖图上仍有作为起点的工序。这一归类把车辆故障划到按工序划界能够给出非空范围的一侧。只有当重调度方法维护了车-任务映射时,这些起点工序才非空;若方法只跟踪工序与机器 (AOR 的常见实现即如此),车辆故障同样会得到空范围。本文不依赖这一点立论,第 6.5 节按维护了该映射的强口径报告 A 类读数。

定义 2 (工序依赖图上的直接集与影响域). 对扰动 $\mathcal{D}$, 令 $T_{\text{direct}}(\mathcal{D})$ 为因机器故障、车辆故障或工序失效而无法按原计划执行的工序集合。对 B 类走廊扰动约定 $T_{\text{direct}} = \emptyset$ 。在 G_T 上从 T_{direct} 出发,按工件后继与同机后继纳入深度至多 θ 的工序,记为 $T_{\text{impact}}(\mathcal{D}, \theta)$ 。

定义 3 (作为起点的占用). 起点集 $\text{Seeds}(\mathcal{D}) \subseteq R$ 按扰动类型给出,一律只取 $r.t^e > t_{\text{now}}$ 的占用。

- (a) **走廊阻断/降速** $\mathcal{D} = (e, [t_a, t_b]): \{r : r.e = e, [r.t^s, r.t^e] \cap [t_a, t_b] \neq \emptyset\}$ 。多处短暂阻断时取各窗起点占用之并。两者共用这条起点规则(原排程按时长未缩放的 τ 写出,与窗重叠的占用都已失效),故划定须改写的占用时二者给出同一个预约影响集——第 6.5 节据此说明为何不把二者的划界读数算作两次独立验证;改路时二者已分开,降速只缩放与窗重叠的那一段进度,而不再写成整段禁行,也不再把有交的穿越整段乘倍率。
- (b) **车辆故障** $\mathcal{D} = (k)$:该车在 t_{now} 之后的全部占用 $\{r : r.k = k\}$ 。
- (c) **机械臂故障** $\mathcal{D} = (m, F)$, F 为该机上尚未完工的工序:对每个工件取 F 中最小工序号 $i_{\min}(j)$,起点占用为各工件自该工序起(含)的全部未来占用。之所以沿工件链向后取而不只取该工序自身,是因为进站运输可能已在 t_{now} 前结束,若只取自身,预约影响集会短于工序依赖图上的影响域,对照即不公平。

命题 1 (B 类上工序依赖图给出空范围). 设 $\mathcal{D}$ 为走廊阻断。若采用定义 2 对 B 类的约定,则 $T_{\text{direct}}(\mathcal{D}) = \emptyset$,而对任意有限 $\theta \geq 0$ 有 $T_{\text{impact}}(\mathcal{D}, \theta) = \emptyset$ 。若此外 $\text{Seeds}(\mathcal{D}) \neq \emptyset$,则 S^0 在 $\mathcal{D}$ 下不可行。

证明. 由定义 2,走廊阻断不使任何机器、车辆或工序无法按原计划执行,故 $T_{\text{direct}} = \emptyset$ 。 θ -步纳入的起点为空,故 $T_{\text{impact}} = \emptyset$ 。若存在 $r \in \text{Seeds}(\mathcal{D})$,则 r 的占用区间与禁行窗相交。将 $\mathcal{D}$ 解释为该走廊上的强制占用后, r 与禁行窗在同一排他资源上时间重叠,违背半开时窗无冲突条件,故 S^0 不可行。□

命题 1 是问题层面上的反例。前半句由定义 2 对 B 类的约定直接得到 (工序依赖图按定义不含走廊占用之间的让行关系,因而覆盖不到走廊阻断); 非平凡的是后半句——起点占用非空时原排程已经不可行,因而「工序依赖图上范围为空白 $\Rightarrow$ 无需重调度」在用时间窗预约走廊的设定下不成立。该反例不依赖任何修复算法;实验第 6 节给出算例规模。

3.4 恢复问题

问题 1 (按预约表划定范围的有界恢复). 给定可行排程 S^0 、扰动 $\mathcal{D}$ 与时刻 t_{now} , 求 S' , 使得: (i) S' 在 $\mathcal{D}$ 下通过无冲突校验; (ii) 未纳入改写范围的占用,其走廊、时段与车辆尽可能与 S^0 一致; (iii) 算法耗时受预算约束。次级目标为完工时间 $C_{\max}(S')$ 。

求解拆为两步:先划定允许改写的占用集合 $U \subseteq R$; 再只对 U 内运输改路,集外占用保持原计划。本文用 $\text{Cl}(\text{Seeds})$ 给出 U ; 对照则用工序依赖图上的影响域诱导出预约子集。

4 时空预约影响闭包

4.1 预约依赖边

定义 4 (预约依赖图). 顶点为 R 中的走廊占用。有向边 $r \rightarrow r'$ 表示「 r 失效可能迫使 r' 改路」,包括:

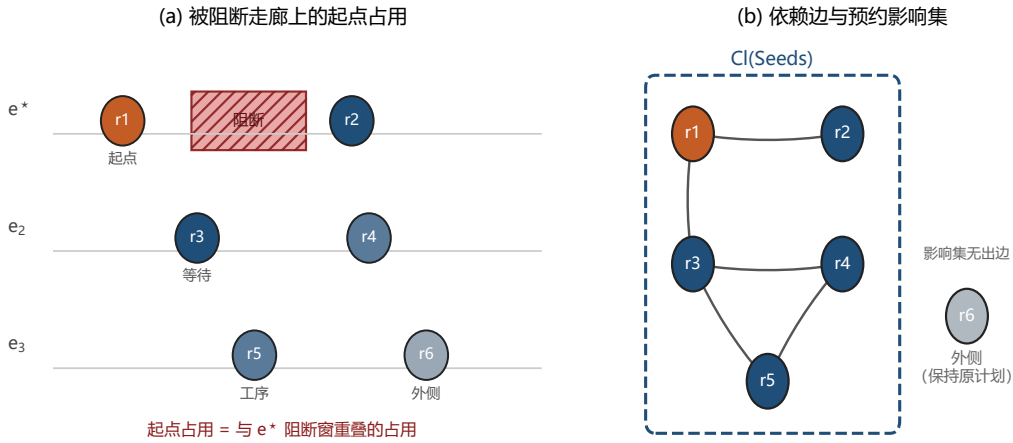


Fig. 3. 预约影响集的构造示意。(a) 与走廊阻断窗重叠的占用作为起点; (b) 沿让行、接续边取传递闭包得到 $CI(Seeds)$, 集外占用保持原计划。

- (1) **让行边**: 同走廊、异车、时间重叠或半开区间接壤 ($r.t^e = r'.t^s$), 且先进入者指向后进入者; 接壤在排他语义下合法, 但前者延后则后者必须让路;
- (2) **同车后继**: 同一车辆按时间序相邻的两条占用;
- (3) **同工件后继**: 同一工件工序 i 的占用指向工序 $i+1$ 的占用;
- (4) **同机后继**: 同一机器时间轴上相邻两道工序所诱导的占用之间的边。

由此得到 $G_R = (R, E_R)$ 。四类边的示意图见图 2(b)。同工件后继连的是相邻工序所诱导的占用, 同机后继连的是同一机器时间轴上相邻两道工序所诱导的占用; 二者都以段为端点, 与第 3.1.1 节的粒度一致。

定义 5 (STRC). 对起点占用集 $S \subseteq R$ 、视界 H 与时刻 t_{now} , 记决策时刻尚未结束、且开始时刻早于视界的占用为

$$R^\circ = \{r \in R : r.t^e > t_{\text{now}}, r.t^s < H\},$$

并令 G_R° 为 G_R 在 R° 上的诱导子图。预约影响集 (时空预约影响闭包) 定义为

$$CI(S) = \{v \in R^\circ : \exists s \in S \cap R^\circ \text{ 使 } s \rightarrow^* v \text{ 于 } G_R^\circ\},$$

即从尚未结束的起点占用出发, 沿出边可达的全部尚未结束的占用 (含起点自身)。若以邻接表表示 G_R° , 则可用队列在 $O(|R^\circ| + |E_R^\circ|)$ 时间内算出。

图 3 给出示意: (a) 阻断窗与作为起点的占用; (b) 依赖边与预约影响集的外壳——集外占用不是集内占用的出边后继 (命题 2)。

按工序依赖图划界的对照, 把 T_{impact} 映射为允许改写的占用: 凡任务标签属于该影响域且 $t^e > t_{\text{now}}$ 的占用。由命题 1, B 类上该集合为空。按预约影响集划界时取 $U = CI(Seeds)$ 。实验表中二者分别记为 R1 与 R2。

4.2 包含性

命题 2 (结构闭合性). 对任意 $S \subseteq R$, 在 G_R° 中不存在边 $u \rightarrow v$ 满足 $u \in \text{Cl}(S)$ 且 $v \in R^\circ \setminus \text{Cl}(S)$ 。因而, 从 $\text{Cl}(S)$ 出发沿出边无法到达影响集外、尚未结束的占用。

证明. 若存在这样的边 $u \rightarrow v$, 则由 $u \in \text{Cl}(S)$ 知存在尚未结束的起点占用 s 使 $s \rightarrow^* u$ 。拼接 $u \rightarrow v$ 得 $s \rightarrow^* v$, 故按定义 5 应有 $v \in \text{Cl}(S)$, 矛盾。□

命题 2 说明「局部」对应一个图论上闭合的对象: 影响集外的占用不是任何集内占用的直接后继。这并不声称 G_R 已枚举物理世界中一切可能耦合——边集是建模选择; 一旦边集固定, 闭合性就是定义的推论, 而不是启发式半径。

注 2 (预约影响集与最小允许改写集合). 结构闭合性说明影响集足够大: 集内占用没有指向集外的依赖边。它并不说明影响集不过大。理由有三。

其一, **最小允许改写集合依赖于所用的修复程序, 而预约影响集不依赖**。称集合 U 为最小, 须指明「在何种修复能力下 U 足以恢复可行」。允许改派与改序时的最小 U , 与只允许改路时的最小 U 不是同一个集合。预约影响集由 G_R 唯一确定, 与修复程序无关——因而在定义上不绑死某一启发式, 也无法同时成为那种依赖修复程序的最小集。

其二, **四类边在充分方向仍是过近似; 必要方向上, 已知的那条接壤通道已经补上**。让行边按「同走廊、异车、重叠或接壤」建立, 只要重叠即连边, 而不判断后者是否真的只能靠等待前者通过——若该处另有旁路, 后者改道即可, 本不必纳入允许改写的集合; 同车后继边同样可能把延误传不到的链尾一并纳入。这两处仍使影响集偏大。接壤一度是反方向的漏边: 半开区间首尾相接合法, 旧规则不连边, 但前者延后则后者必须让路。现已并入让行边, 差分试探在 50 对上泄漏为 0(0/50; 第 7.1 节)。这关闭了这一条已观测到的通道, 不是边集完备的证明。

其三, **实测上影响集并不小**。第 6.5 节报告, 走廊阻断下预约影响集平均覆盖 t_{now} 之后尚未结束占用的约 90%; 换言之, 相对「把决策时刻尚未结束的占用全部纳入改写」这一平凡上界, 影响集少纳入的不足一成。有界修复真正省下的是不改写已经结束的占用与不重开搜索, 而不是把未来切得很小。这一点在第 6.8 节与局限一节中再次点明, 不应被「局部」「有界」这两个词读成「只动很小一块」。

「不足一成」是允许改写集合的规模之比, 不等于改写比例之差。第 6.9 节把上述平凡上界作为一条独立对照(RA)实测后发现, 正是这不足一成的差别, 把改动比例从 0.604 降到 0.540(逐格 31/18/1, $p = 8.8 \times 10^{-7}$), 而算法耗时与完工时间几乎不变。

因此, 本文给出的是由结构唯一确定、可复现、足以恢复可行的允许改写集合, 并附带闭合性; 逼近最小允许改写集合是另一个问题, 列为后续工作(第 7.2 节)。

命题 3 (外侧字段不变的充分条件). 设允许改写的占用集合 $U = \text{Cl}(S)$, 并设第 1 级改路按第 5.1 节执行: (i) 禁行窗已写入路由层; (ii) 对每个 $r \in R^\circ \setminus U$, 将其原字段 $(e, [t^s, t^e], k)$ 预提交到新预约表且预提交成功; (iii) 仅对 U 内运输调用路由器生成新占用; (iv) 最终排程通过无冲突校验。则在恢复后的预约表 R' 中, 每个集外任务标签对应的占用, 其走廊、时段与车辆与 S^0 一致。

证明. 由 (ii), 集外占用在重放开始前已按其原区间写入新表, 且未与禁行窗冲突 (否则预提交失败)。步骤 (iii) 只为 U 内任务提交新占用, 不删除或改写已提交的集外区间。因此, 对集外任务标签, 新表中的 $(e, [t^s, t^e], k)$ 与预提交值相同, 亦即与 S^0 相同。校验通过仅排除新占用与既有占用冲突, 不改变已提交字段。□

注 3 (前提 (ii) 与 (iii) 各自会怎样失效). 若预提交因「集外占用与禁行窗重叠」失败, 则 S 未能覆盖全部与阻断窗重叠的占用, 起点集不完备。若改路因运输-工序衔接失败而触发第 5.2 节扩大范围, 则新的允许改写集合 $U' \supsetneq U$, 命题 3 只对最终未纳入改写的集外占用成立。扩大范围以可行性优先于改写尽可能少, 实验中单独报

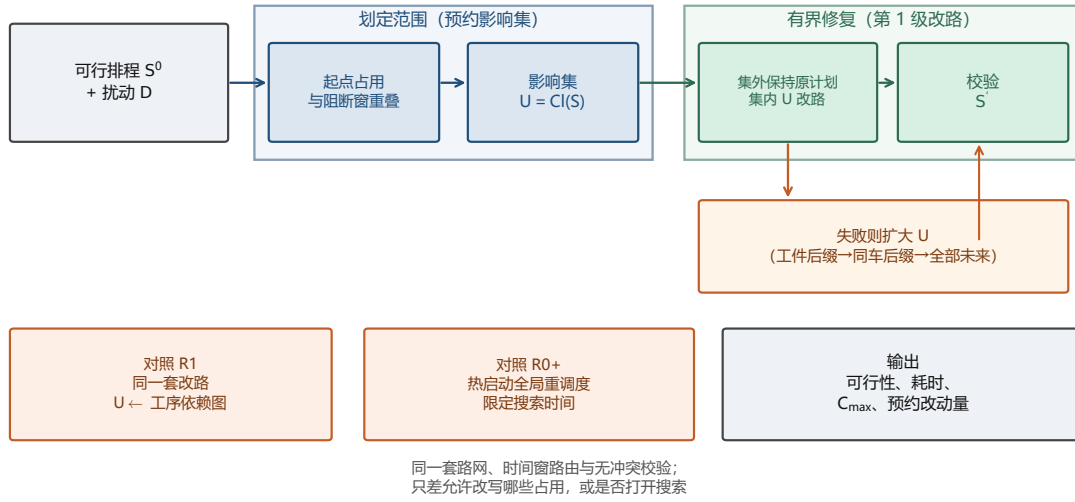


Fig. 4. 按预约影响集划界的有界恢复。上排:由起点占用得到影响集并改路;中排:失败后扩大允许改写的范围;下排:按工序依赖图划界的对照与热启动全局重调度对照,以及输出指标。

告。还有一种失效不在命题的可控范围内,但在实现上容易发生:前提 (iii) 要求路由器只 为 U 内的运输生成新占用。若实现以比占用更粗的单位(例如整道运输、整个工序)决定是否改路,则 U 外的占用会被顺带改写,命题的结论随之不成立——不是命题错了,而是前提没被满足。注 4 记录了本文实现一度按整道运输改路的情形。

4.3 结构可预测性

预约影响集的规模不是可调参数,而是由排程与路网决定的量:它随算例规模稳定上升(第 6.2 节)。因而「局部」是可以测量的,而不是人为设定的半径。

至于哪种结构对应更大的影响集,实验给出的是否定读数。本文先考察一类只由路网算出、不随当前排程变化的指标:把装卸点与车间其余部分隔开所需切断的最少走廊数,即装卸点最小割,连同漏斗占比;下文把这一族称作静态割。影响集规模不由静态割决定。在自建的同规模布局上,不同边界阻断协议下的相对规模或落在很窄的带内、或虽有方向但各随机种子上的取值高度重叠;换用一批不由本文设计的路网后,装卸点最小割与漏斗占比取值恒定,而影响集占比在路网不变、仅改机器落位时即大幅移动——一个恒定的量不可能解释一个变动的量(第 6.7 节)。这条否定只针对「静态割能解释影响集规模」这一命题,并不等于「结构与影响集规模无关」。可得的公开路网数量有限,后者尚不可判。下一个候选刻画应是随排程而变的量,而非纯拓扑量(第 7.2 节)。

5 预约影响集上的有界修复

图 4 总览本节流程:先按预约影响集划定允许改写的占用集合 U ,再只对集内运输改路、集外保持原计划;仍不可行则扩大 U 再试。按工序依赖图划界的对照只替换 U 的来源;全局重调度对照则在限定时间内重新搜索。各方法共用同一套走廊路网、时间窗路由与无冲突校验。

算法 1 第 1 级改路重放 REPLAYREROUTE

Require: 排程 $S^0 = (x, \pi, y, R)$, 扰动 $\mathcal{D}$, 允许改写集合 U , 时刻 t_{now}

Ensure: 排程 S' 或失败

- 1: 将 $\mathcal{D}$ 写入路由层 ▷ 阻断:禁行窗;通行变慢:重叠段 τ
- 2: 清空 $t^e > t_{\text{now}}$ 的占用,得到空表 R'
- 3: **for** 每条集外占用 $r \notin U$ 且 $r.t^e > t_{\text{now}}$ **do**
- 4: 将原字段 $(e, [t^s, t^e], k)$ 预提交到 R'
- 5: **if** 与阻断窗冲突 **then**
- 6: **return** 失败 ▷ 起点占用未覆盖与窗重叠的占用
- 7: **end if**
- 8: **end for**
- 9: **for** 每条 $r \in U$,按原扫描序 **do**
- 10: 已走完的走廊腿预提交;从该车间在 t_{now} 的节点接起
- 11: 在当前 R' 上调用时间窗路由,提交新占用 ▷ 不改写已经结束的占用
- 12: **end for**
- 13: 按 π 推进各机器空闲时刻与各工件就绪时刻
- 14: 独立校验:无冲突、无禁行窗残留、通行变慢的重叠段时长已吸收、集外字段未改
- 15: **if** 校验通过 **then**
- 16: **return** $S' = (x, \pi, y, R')$
- 17: **else**
- 18: **return** 失败
- 19: **end if**

5.1 允许改写、保持原计划与第 1 级改路

给定允许改写的占用集合 U ,第 1 级改路保持原车辆指派与机器指派,按原扫描序重放。算法 1 写出这一原语;算法 2 在它外面套预约影响集,失败则扩大 U 。

预约影响集是 G_R^o 上一次广度优先,时间 $O(|R^o| + |E_R^o|)$ 。一次 REPLAYREROUTE 对 $|U|$ 个运输段各调用一次时间窗最短路;实测耗时由后者主导(第 6.4、6.9 节)。该级不打开种群搜索。第 6.4 节的对照固定算法 1,只替换 U 来自工序依赖图或预约影响集,用以把可行率之差归因于划界对象。

5.2 失败后扩大允许改写的范围

扩大允许改写的范围,本身已有成例。Local Rescheduling [8] 按时间窗逐步放宽直至可行;AOR 则按工件后继与同机后继纳入后续工序(第 2.1 节)。本文沿预约依赖边扩大 U ,而不是沿时间窗。这些步骤用来落实影响集:改路失败时仍先扩大须改写的占用,而不是立刻改开全局搜索。第 6.1 节说明边界消融实验为何必须关闭这一步。

当集外占用保持过紧时,算法 1 可能因运输-工序衔接等校验失败。此时不立刻改开全局搜索,而按下列顺序扩大 U 并重试同一套改路程序: (i) $U \leftarrow U \cup \{\text{当前 } U \text{ 内各工序的同工件后继、尚未结束的占用}\}$; (ii) $U \leftarrow U \cup \{\text{所涉车辆的同车后继、尚未结束的占用}\}$; (iii) $U \leftarrow \{r \in R : r.t^e > t_{\text{now}}\}$ (把决策时刻尚未结束的占用全部纳入改写)。扩大范围后仍是确定性的单遍改路。实验表明这一步可将可行率拉至全样本,算法耗时仍保持在毫秒至十余毫秒量级(第 6 节)。耗时随扰动比例 φ 并不单调(表 11):比例小时初始影响集偏紧、集外保持的占用偏多,更常需要扩大范围;比例大时初始 U 已较松、往往一轮成功,但单轮要改路的运输更多。两种效应方向相反,故本文只报告耗时不随 φ 崩溃,不声称它随 φ 有可预测的走向。

算法 2 基于预约影响集的有界恢复(第 1 级改路 + 扩大范围)**Require:** 排程 S^0 , 扰动 $\mathcal{D}$, 时刻 t_{now} **Ensure:** 排程 S' 或失败1: $S \leftarrow \text{Seeds}(\mathcal{D}); U \leftarrow \text{Cl}(S)$ 2: **for** 轮次 = 0, 1, 2, 3 **do**3: $S' \leftarrow \text{REPLAYREROUTE}(S^0, \mathcal{D}, U)$ 4: **if** S' 可行 **then**5: **return** S' 6: **end if**7: 按扩大范围的规则更新 U

► 工件后缀 / 同车 / 全部尚未结束

8: **end for**9: **return** 失败**5.3 与全局重调度的对照**

全局重调度对照在相同禁行约束下、在限定时间内以原排程为起点重新搜索(记 R0+)。内部把 (x, π) 编成一条染色体;一次解码指:按该染色体的指派与扫描序,在装了禁行窗的预约表上从头为全部运输调用第 3.1.2 节的时间窗路由,得到新的 R 与 C_{max} 。R0+ 重复解码并允许改染色体;第 6.9 节的 RD 只解码一次且不改染色体。R0+ 的解码不改写已经结束的占用:这些占用预提交,只从 t_{now} 接着排未开始的工序(第 6.8 节)。RD 与 R0+ 走同一套固定前缀解码,只解码一次且不改染色体。对照取热启动而非从零搜索,以免丢掉原排程已有的指派与路径信息。R0+ 可改派与改序,因而更可能缩短 C_{max} ,但算法耗时由预算决定。

有界修复在完工时间上并不优于 R0+。第 6.8 节表明:R0+ 在最小预算下已占优,两条曲线不相交。有界修复的长处是算法耗时:响应须在毫秒量级时,它能恢复可行;允许秒级停机时,应用 R0+ 去缩短 C_{max} 。集外占用的走廊、时段与车辆保持原计划,只在命题 3 的条件下保证,实验中作部分观测。

只与 R0+ 对照会留下一个问题:毫秒量级那一档里,是否根本不需要按影响集划界?把「不按影响集限制范围」的做法直接实现出来即可回答——全局右移不判断哪些占用受影响,只把未完成占用整体后推到阻断结束之后;而「重算但不按影响集划界」这一端,耗时更短,形态是按原染色体重解码一遍。两者都在毫秒量级,构成有界修复在同一耗时档位内的对照,读数在第 6.9 节。

算法 2 与图 4 对应:预约影响集给出 U , 第 1 级改路,失败则按工件、车辆、全部尚未结束占用的顺序扩大 U 。

6 实验**6.1 协议**

路网、路由与算例. 下层路网、时间窗路由与独立校验器固定。扩样矩阵使用五个算例:小例 example_3x3x2、拥堵例 congested_8x4x4,以及三个 S8x4x4 布局 high、funnel 与 mid。后三者同规模、同柔性,只差走廊拓扑,因此它们之间预约影响集规模的差异可归因于拓扑而非问题尺寸。随机种子取十个,即 42、7、2024、99、123、13、1、777、31415 与 8,共 50 个算例-随机种子对。初始排程由第 3.1.2 节的时间窗路由按扫描序重放得到,并通过独立无冲突校验;同一算例-随机种子对下各对照方法共享该初始解。本文不依赖这份 S^0 的搜索来源,只要求它在扰动前可行。

第二批:布局出外外部化. 上面五个算例的布局都是本文设计的,其中三张同规模布局还同属哑铃一族(彼此只差装卸点出口与中段的并行通道数,即只差容量、不差几何)。为了让布局不再由本文决定,另跑一批 5 个算例:路网的网格尺寸、装卸站与机器落位、断掉的边这三项逐项转录自 Lyu 等 [10] 附录 A 的布局图,转录结果由脚本与 van Os 配套工具集所给的机读编码逐字段对账。机器台数因此由布局决定而非由本文选定,为 4 至 8 台。随

机种子仍取同样十个,合计 50 个算例-随机种子对。两批算例逐参数同口径:工件数、AGV 数、每工件工序数、 H 、 F 以及运输/加工时长比的标定目标一律相同,于是两批之差可以干净地归因给布局出处。

该附录共给出六张布局(3 至 8 机)。3 机那张未用:柔性度 F 的含义是平均候选机数占机器总数之比,本文固定 $F = 0.6$,在 $|M| = 3$ 时给出平均候选机数 $F \cdot |M| = 1.8 < 2$,无法在满足「每道工序至少两台候选机」的前提下达到该柔性目标;为一张布局放宽 F 等于多引入一个变量,故宁可少用一张。

其余五张只落在 2 张不同路网上,须先说明:4 机与 5 机两张共用同一张 4×4 网格(去掉同一条边),6、7、8 机三张共用同一张 5×5 网格(去掉同两条边);五者彼此的差别在于机器台数与落位,而非路网几何。这是原始发布件本身的组织方式,不是本文的取舍。因此第 6.7 节读这批数时,路网几何这一维的独立样本是 2 而不是 5,而机器落位这一维有 5 个;下文的论断严格按这个划分来下。

有三处口径必须先声明清楚。其一,逐段行驶时间不是原始数据。Lyu 只为单个示例算例发表过逐段时长,且其取值并不均匀;附录 A 那批布局的逐段时长从未发表。故这里按「所有边等权」补齐——它与 van Os 模型里「单步时长为常量」的假设在结构上一致,只是时间单位的标度不同。因此本批结果不可与 Lyu 或 van Os 的参照值作任何比较;它提供的只是一批不由本文设计的拓扑,而不是一个对标集。其二,装卸点做了合并。原布局有分离的装货站与卸货站,本文只有一个装卸点,故取车辆起始处的装货站充当该点,卸货站退化为普通网格节点。其三,工件数据没有一并借用。已实测的公开族里平均运输时长只占平均加工时长的百分之几,运输在其上几乎不构成争用;若把工件数据也搬进来,走廊争用连同以它为对象的整套结论都会退化到观察不到。外部化的只有路网。同一工具集另附的一族布局(其目录标注为 Liu 2023)未纳入本批:这些布局的数据声明允许对角移动,而本文的走廊是四邻接的;接受对角移动要改的是下层路由而不是算例。

扰动。除非另注, $t_{\text{now}} = 0.35 C_{\text{max}}(S^0)$ 。E1-E3/E5 在繁忙走廊上施加单窗阻断;E6 在同一 t_{now} 上另加三类扰动;规模扫描将未来工序按最早未来预约排序,取前比例 φ 的预约时窗作为微阻断(不合并)。

对照方法。R1:按工序依赖图上的影响域划界 + 第 1 级改路,全文取 $\theta = 2$ (该值只影响 A 类扰动下 $|U_{R1}|$ 的大小;由命题 1,B 类上任何有限 θ 都给出空集); R2/STRC:按预约影响集划界 + 第 1 级改路(规模扫描打开失败后扩大范围;E3/E5 关闭这一步); R0+:以原排程为起点、在限定时间内重新搜索的全局重调度; RS:全局右移,不按影响集划界且不改路(第 6.9 节); RD:原染色体重解码,即本实现里耗时更短的一次全局重算(同节); RA:不按影响集限制范围、但保留改路——把 t_{now} 之后尚未结束的占用全部纳入改写,其余与 R2 逐项相同,用于把预约影响集这一项单独隔离出来(同节)。这五条对照按「是否划界 $\times$ 重算多少」铺开:R1 与 R2 只差划界对象,RS 不划界而整体后推,RD 与 R0+ 则重算全局而不按影响集划界,只差搜索多少代。

一处必须写清的口径。E3 中必须关闭失败后扩大范围。若开着,R1 会靠「一路扩到把尚未结束的占用全部纳入改写」而偶然成功,测到的就不再是划界对象的差别,而是扩大规则的兜底能力。扩大范围本身有效(规模扫描中它把可行率拉满),但它与被测变量正交,故在边界消融中一律关闭。该端点并未因此被回避:它就是第 6.9 节的 RA,那里作为独立对照逐格测了它的代价。

两批算例各自承担什么。两批不是同一组证据的两次采样,它们回答的问题不同,故本章各节只由其中一批支撑。主批次回答现象是否存在、机制是否干净:B 类扰动下按工序依赖图划界得到空范围而原排程不可行(第 6.2 节)、预约影响集对该范围的包含与集外字段的不变性(第 6.3 节)、同一套改路程序下只替换划界对象的对照(第 6.4 节)。这三件事都要求对算例参数有完全的控制权,尤其是运输/加工时长比——它决定走廊争用是否成立——而公开族给不了这种控制权(第 7.1 节局限第 4 条)。第二批回答的是另一个问题:这些现象是否只在本文自选的布局上成立。它因此只用于复现,不产生新的性能读数(第 6.12 节)。

两处需要点明。其一,第二批用来检验现象是否只在本文自选布局上成立,并收窄结构可预测性的结论(第 6.7 节)。其二,只有 E4 一格必须两批合读:主批次的三张同规模布局同属一族、只差容量不差几何,在它们身上"指标太粗"与"算例集几何差异不足"这两种解释混在一起;而外部布局若没有逐参数同口径的自建对照,也无从判断其影响集占比的变动幅度算不算大。其余各节的结论由主批次单独成立,第二批只增强其外部效度。

表 3. E1 走廊阻断下工序依赖图给出空范围(每算例 10 个随机种子,共 50 对)。|R| 为全表占用数,CI/|R| 为预约影响集占全表的比例。均值取多种子平均。后三行同规模同柔性,只差走廊拓扑。

算例	随机种子	C1 通过	均 R	均 Seeds	均 CI	CI/ R
example_3x3x2	10	10/10	28.6	5.6	16.9	0.59
congested_8x4x4	10	10/10	85.0	15.0	48.4	0.57
S8x4x4 (high)	10	10/10	150.5	15.5	88.3	0.59
S8x4x4 (funnel)	10	10/10	149.1	15.5	85.4	0.57
S8x4x4 (mid)	10	10/10	147.6	15.0	82.8	0.56
合计	50	50/50	—	—	—	—

表 4. E2 包含性(10 个随机种子/算例)。E2a 为结构闭合(泄漏 = 0), E2b 为集外占用的走廊、时段与车辆逐条不变。

算例	E2a 闭合	第 1 级可行	E2b 集外逐条不变
example_3x3x2	10/10	10/10	10/10
congested_8x4x4	10/10	10/10	10/10
S8x4x4 (high)	10/10	10/10	10/10
S8x4x4 (funnel)	10/10	10/10	10/10
S8x4x4 (mid)	10/10	10/10	10/10
合计	50/50	50/50	50/50

6.2 E1:工序依赖图给出空范围

表 3 与图 5(左)汇总命题 1 的扩样结果: 50/50 样本上 $T_{\text{impact}} = 0$ 、起点占用与预约影响集非空、阻断后原排程不可行, 结构泄漏均为 0。

按命题 1 之后那段话的提醒,读这张表时重量应压在后半句上。 $T_{\text{impact}} = 0$ 这一列是定义 2 对 B 类约定的推论,它把「工序依赖图按定义覆盖不到走廊阻断」显示出来,但本身不是发现。非平凡的是feasible_after_block 一列恒为假——排程确实已经不可行。两者合起来才构成反例。

预约影响集的规模随算例规模上升(16.9 → 48.4 → 80 余),但CI/|R| 在五个算例上稳定在 0.56–0.59,未能区分三个 S8x4x4 布局。这与第 6.7 节的结论一致:实验没有给出能可靠预测影响集规模的结构指标。

6.3 E2:包含性

结构包含性(E2a)在 50/50 样本上成立(泄漏 = 0),与命题 2 一致。第 1 级修复(不扩大范围)在 50/50 样本上可行。集外占用的走廊、时段与车辆逐条不变(E2b / 命题 3 的观测形式)同样在 50/50 样本上成立(表 4)。

需要区分 E2a 与 E2b 说的不是一回事,满通过并不使命题 3 升格为充要。E2a 是图上的性质,由命题 2 保证,满通过是应当的;若不满通过,说明实现与定义不符。E2b 是修复之后的性质,它检验的是命题的四条前提在本文协议下是否成立——而不是这些前提可以省略。特别地,前提 (ii) 要求集外占用逐条预提交成功;若某条集外占用与禁行窗重叠,预提交即失败,那说明起点占用集不完备(注 3),此时命题不适用。本文的实验落在前提成立的一侧,不等于任何设定下都会落在那一侧。

注 4(一处曾经把实现缺陷读成边集缺陷的教训)。这一栏在早前的批次上只有 24/50。当时的解读是「依赖边集未能覆盖若干弱耦合」,并据此把加细边集列为首要后续工作。复核后该解读是错的。允许改写的是一个占用集合,而重放时是否改路却按工序判定——同一工序的空载段与满载段只要有一段落在影响集内,整道运输就被重规划。于是当只有满载段进入影响集时,同工序的空载段也被改写,而它的任务标签并不在允许改写的集合内,遂被记为集外漂移。26 个不通过格上的 84 条漂移占用无一例外都是这种同工序空载段,其中 71 条(85%)在

表 5. E3 边界消融(不扩大范围;5 算例 $\times$ 10 个随机种子)。miss_B 指该格上 R1 的允许改写集合为空而 R2 非空。

算例	miss_B	R1 可行	R2 可行	均 $ U_{R2} $	R2 耗时/ms
example_3x3x2	10/10	0/10	10/10	16.9	0.66
congested_8x4x4	10/10	0/10	10/10	48.4	2.84
S8x4x4 (high)	10/10	0/10	10/10	88.3	3.40
S8x4x4 (funnel)	10/10	0/10	10/10	85.4	5.09
S8x4x4 (mid)	10/10	0/10	10/10	82.8	3.96
合计	50/50	0/50	50/50	—	—

t_{now} 之前就已执行完毕——改写它同时违反「不改写已经结束的占用」。改为按段判定(已执行完或未纳入改写的空载段沿用原计划,只改路真正落在允许改写集合内的段)之后,漂移归零。换言之,此前测到的不是边集对物理耦合的覆盖度,而是命题 3 前提 (iii)「仅对 U 内运输调用路由器」在实现上没有被真正满足。记下这一条,是因为它示范了一种容易发生的误判:把实现与定义的粒度错配,读成模型对世界的刻画不足。

同一类错配还剩一层,而 E2b 查不到它。按段判定之后,一个跨越 t_{now} 的运输段仍是整段重规划:只要它有一条走廊腿的结束时刻落在 t_{now} 之后,整个段的任务标签就进入允许改写的集合,于是它在 t_{now} 之前已经走完的那几条腿也被重排——等于把车退回该段起点重走一遍,违反「不改写已经结束的占用」。E2b 查不出这一情形,因为这些腿的任务标签在影响集内,本就不在集外字段的比对范围里。用一项独立的「已结束占用不可改写」审计(逐条比对 $t_{\text{end}} \leq t_{\text{now}}$ 的占用)才测出来:50 个格子里有 36 个存在此类改写,合计 86 条。把粒度再细一层——按走廊腿切分,已走完的腿原样保留,改路只从车在 t_{now} 时实际所处的节点往后接——之后该审计在全部 50 个格子上归零,且各项通过率不变、 C_{max} 在 6 个格子上变好、无一格变差。本文报告的是修正后的读数。

6.4 E3:边界消融

表 5 与图 5(右)固定第 1 级改路并关闭失败后扩大范围。B 类上 50/50 样本 R1 的允许改写集合为空且不可行,R2 全部可行。

这是本文最强的一组对照,但它的说服力来自只差一项而不是来自数字大:同一套改路程序、同一份初始解、同一个 t_{now} 、同一批随机种子、同一段阻断窗,唯一的差别是允许改写哪些占用。因此 0/50 对 50/50 这个反差不能归因于改路启发式的优劣——R1 侧根本没有触发任何改路,它的允许改写集合是空的,第 1 级改路对空集是恒等操作,于是原排程原封不动地留在被阻断的走廊上。换句话说,R1 的失败方式不是「修得不好」,而是「判断为无需修」。算法耗时中位数 3.2 ms、最大 12.8 ms。

6.5 E6:扰动类型与边界定义的交叉

E1–E3 只用了走廊阻断一种扰动。本小节把扰动类型这一维展开,在同一 50 对上另加走廊通行变慢、车辆故障与机械臂故障三类,给出表 6。

先说测什么。划界一侧四类都报:工序依赖图上的影响域有多大、预约影响集有多大、后者是否包含前者。修复一侧四类都开口。B 类仍只改路:阻断把禁行窗写入预约表,通行变慢在窗内按倍率 $\tau_{\text{mult}} = 2$ 降低进度、窗外仍走原 τ ,走廊仍可走。A 类额外换车或改派,不改扫描序;是否扩大允许改写的范围,对它们不改变动作集合。

四点读数。第一,A/B 二分在 200 次划界测量上无一例外:两类 B 扰动按工序依赖图划界全部为空,两类 A 扰动全部非空。第二,在全部 100 次 A 类测量上 $U_{R1} \subseteq C1$ ——这说明改用预约影响集划界在 A 类上不会丢掉工序依赖图已纳入的占用。包含在 87/100 上是严格的,其余 13 格两者恰好相等(此时影响集沿依赖边没有走出工序依赖图影响域诱导的那个集合),不是包含失败。代价是允许改写的集合平均变大:车辆故障 31.1 $\rightarrow$ 60.6,机械臂故障 42.8 $\rightarrow$ 65.9。结合第一点,预约影响集在这四类扰动上是工序依赖图划界的一致加强,而非另一种取舍。第

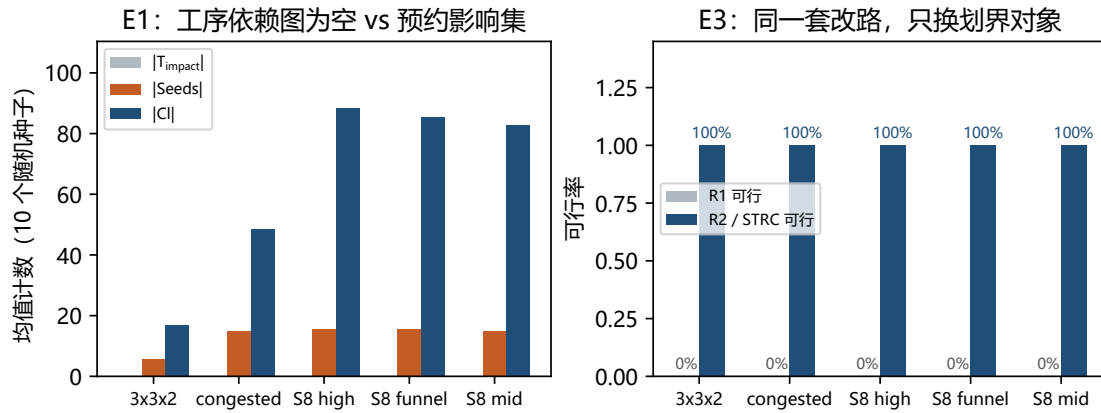


Fig. 5. E1/E3 扩样可视化(5 算例 $\times$ 10 个随机种子均值)。左:工序依赖图上的影响域为空,而起点占用与预约影响集非空;右:同一套改路、只换划界对象时 R1 可行率为 0、R2 为 100%。

表 6. E6 扰动类型 $\times$ 划界对象(50 对/类,共 200 次划界测量)。 $|R^o|$ 为 t_{now} 之后尚未结束的占用数。「R1 为空」即按工序依赖图划界判该扰动无需重调度的格数。未列不扩大范围:B 类为第 1 级改路,A 类为换车或改派。

扰动	类	R1 为空	均 $ U_{R1} $	均 $ CI $	$CI/ R^o $	R2 可行
走廊阻断	B	50/50	0.0	64.4	0.92	50/50
走廊通行变慢	B	50/50	0.0	64.4	0.92	50/50
车辆故障	A	0/50	31.1	60.6	0.85	50/50
机械臂故障	A	0/50	42.8	65.9	0.94	50/50

三,通行变慢与阻断的划界两行逐格相同,在全部 50 对上影响集规模一致。这不是两次独立的划界验证:起点规则对二者都是「与失效时窗重叠、尚未结束的占用」,故它们按构造给出同一个起点集,进而给出同一个预约影响集。该行证实的是按工序依赖图划界对通行变慢同样为空。第四,修复一侧四类均满。阻断 50/50;通行变慢同口径 50/50。通行变慢一度把与窗有交的穿越整段乘 2,13 格死在「占用偏短」——窗内 2τ 无空档时,进入时刻被拉回窗内却仍按原 τ 落盘;其中 12 格连把尚未结束的占用全部纳入改写也过不了,故不是集外保持原计划把变慢的车卡死。改为只缩放重叠进度之后,第 1 级满通过,不再触发扩大范围。走廊仍可走,与「整段封死再绕路」的物理差别还在,只是不再被整段倍率放大。车辆故障换车 50/50,机械臂故障改派 50/50。A 类不走扩大允许改写范围,开关不改变动作集合。换车与改派是落实 A 类恢复的动作,用来补全覆盖矩阵的修复一侧。

这张表也给出了影响集规模的实测(注 2)。走廊阻断下预约影响集覆盖 t_{now} 之后尚未结束占用的约 92%;相对「把尚未结束的占用全部纳入改写」这一平凡上界只少纳入一成。有界修复真正省下的是不改写已经结束的占用与不重开搜索,而不是把未来切得很小。第 6.2 节按全表统计的 $CI/|R| \approx 0.57$ 与此不矛盾:全表包含 t_{now} 之前已经执行完毕、本就不该改的那一半。两个口径都给出。

6.6 案例甘特

E1-E3 给出的是跨随机种子的计数与可行率。为把「工序依赖图覆盖不到、预约影响集可修」落到时间轴上,图 6 取 example_3x3x2、随机种子 42 的一例走廊阻断,协议同 E2/E3:决策时刻取 $0.35 C_{\text{max}}$,繁忙走廊单窗阻断,第 1 级改路、不扩大范围。

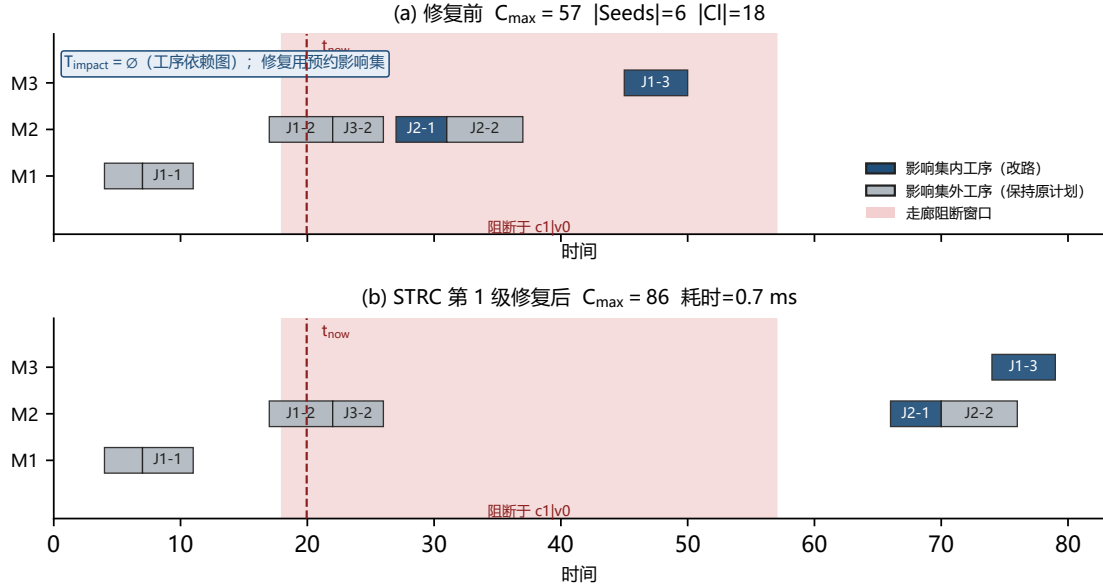


Fig. 6. 案例甘特(example_3x3x2, 随机种子 42)。 (a) 阻断前原排程,按是否属于 CI 着色; (b) 第 1 级修复后。工序依赖图上的影响域为空;蓝色工序对应影响集内运输被改路。

该例上 $T_{\text{impact}} = \emptyset$, 而 $|\text{Seeds}| = 6$ 、 $|\text{CI}| = 18$ 。图 6(a) 为扰动前原排程的机器甘特:蓝色工序的运输占用落入影响集(将被改路), 灰色工序在影响集外(预提交、保持原计划);红色虚线为 t_{now} , 浅红带为阻断时窗。图 6(b) 为修复后:影响集内工序时间轴重排, 排程在阻断下恢复可行, $C_{\text{max}}: 57 \rightarrow 86$, 耗时亚毫秒级。本图只说明统计结论对应何种局部改动形态, 不用于比较完工时间; 完工时间与全局重调度的权衡见随后的 E5 与规模扫描。

6.7 E4:结构(探索)

在 LU 割边阻断下, funnel 与 high 的预约影响集占比随随机种子波动(表 7)。方向上确实是 funnel 更高——中位 0.254 对 0.204, 均值 0.244 对 0.229 ——但两组取值高度重叠: funnel 落在 $[0.180, 0.309]$, high 落在 $[0.156, 0.372]$, 后者的区间把前者整个包住, 且单个随机种子即可反向 (2024 一列上 high 的 0.372 反高于 funnel 的 0.296)。五个随机种子、两个布局, 这个差距不足以支撑任何结论。第 6.2 节按繁忙走廊阻断协议在三个同规模布局上得到的 $\text{CI}/|R|$ 亦落在 0.54–0.57 的窄带内, 同样未能区分布局。

换一批不由本文设计的布局, 这条负结果的成因才分得清。上述三张同规模布局同属哑铃一族, 彼此只差并行通道数, 也就是只差容量、不差几何; 于是 “指标太粗” 与 “算例集本身几何差异不足” 这两种解释在它们身上是混在一起的, 无从分辨。第 6.1 节第二批把两者分开了(表 8)。

结果的方向与预期相反, 而这恰是关键。按第 6.1 节的声明, 这批算例在路网几何上只有 2 个独立样本, 故不能用它来论证 “影响集占比随路网几何变化”; 能论证的是另一件事。在同一张 4×4 路网上, 只把机器台数与落位从 4 机换成 5 机, 影响集占比中位即由 0.512 落到 0.305, 极差 0.207; 同一张 5×5 路网上换三种落位, 极差 0.075。作为对比, 逐参数同口径的三张自建布局——它们是三张不同的路网——只落在 0.446–0.497, 极差 0.051。即: 路网固定、只动机器落位, 影响集占比的变动幅度可以超过换三张不同路网的幅度—— 4×4 那张上是约四倍, 5×5 那张上约一点五倍。两张路网上都成立的是不等号的方向, 倍数本身只有两个样本, 不宜细读。

表 7. E4 预约影响集占比 $Cl/|R|$ (LU 割边阻断;随机种子 42, 7, 2024, 99, 123)。本表算例为 12 工件 / 8 机 / 12 车、运输与加工时长比标定为 4.0。该口径与表 8 不同(后者为 8 工件 / 4 机 / 4 车、比值 1.0、十个随机种子),两表虽共用 funnel/high 这两个布局名, 数值不可跨表比较;表 8 内部才是同口径对照。

布局	42	7	2024	99	123	中位
funnel	0.254	0.181	0.296	0.309	0.180	0.254
high	0.156	0.204	0.372	0.229	0.184	0.204

表 8. E4 布局出外外部化对照(LU 割边阻断,十个随机种子; $Cl/|R|$ 取中位)。全表逐参数同口径,只差路网与机器落位。外部算例只落在 2 张路网上 ($N_1:4 \times 4$ 去一条边; $N_2:5 \times 5$ 去两条边), 组内各行共享同一路网,只差机器台数与落位;"极差"一列按组内给出。要点: N_1 组内路网不变、仅换落位即得极差 0.207,已是三张不同 自建路网之间极差 0.051 的约四倍;而 LU 最小割与漏斗占比在整个外部批 取值恒定,故无从解释这些差异。边权为本文补齐的等权值,故本表不与外部文献的参照值比较。

出外/路网	算例	机器	节点/走廊	LU 割	漏斗占比	$Cl/ R $	组内极差
外部 N_1	LyuL2	4	16/23	2	0	0.512	0.207
外部 N_1	LyuL3	5	16/23	2	0	0.305	
外部 N_2	LyuL4	6	25/38	2	0	0.385	0.075
外部 N_2	LyuL5	7	25/38	2	0	0.422	
外部 N_2	LyuL6	8	25/38	2	0	0.460	
自建	high	4	10/10	2	0.286	0.497	0.051
自建	funnel	4	9/8	1	0.286	0.491	
自建	mid	4	11/12	2	0.286	0.446	

于是只由路网决定的最小割解释不了影响集规模。在这 5 个外部算例上,装卸点最小割 (LU 割) 恒为 2、漏斗占比恒为 0—— 两张路网都把装卸站放在网格角点,而四邻接网格的角点恰有两条出边,这个割因此被钉死。同一工具集另附的那族布局(即因允许对角移动而未纳入本批的一族)把装卸站放在网格边的中点, 其出边数是 3—— 换言之,这个指标在真实布局上只取到极少数几个由"装卸站摆在哪种网格位置"决定的值,并不随几何连续变化。关键在于:这两个指标在上述整段变动中一动不动,而被解释的量已经变了 0.207。一个取值恒定的量不可能解释一个如此变动的量,所以它们连"相关性弱"都谈不上,是无从谈起;第三个指标(每节点走廊数)虽有两个取值,与影响集占比的秩相关为 0。

据此可把读数收紧为:影响集规模不由静态割决定。这一族指标本是围绕哑铃布局那个可调的装卸点咽喉设计出来的,迁到真实布局上就退化成常量,不具备解释力。上面那组读数指出了替代方向:既然路网不变而落位一变,影响集占比就大幅移动, 那么决定影响集规模的不是路网能提供多少条通路,而是需求落在哪里—— 应当刻画的是 t_{now} 邻域内走廊占用的时序密度,而不是静态拓扑的连通度(第 7.2 节)。

6.8 E5:与全局重调度的权衡

本小节原拟找一个交叉点:预算紧时有界修复占优,预算放宽后热启动全局重调度在 C_{max} 上反超。该交叉点不存在:有界修复的完工时间在全部预算点上较差。

表 9 与图 7 在两个算例上对随机种子 {42, 7, 2024} 扫描 R0+ 预算 {0.2, 1, 2} s,共 $2 \times 3 \times 3 = 18$ 个预算点。R0+ 即使在最小预算 0.2 s 下也已优于 R2;加大预算几乎不再拉开(小例三档持平,拥堵例再降约 4)。在全部 18 个点上无一例外。

有界修复的完工时间全区间较差;算法耗时低两至三个数量级。与 R0+ 的改动比例则已同在 50%–66% 量级(少改写须看第 6.9 节与 RA 的对照:两边改写比例已经接近,不能从本表读出稳定性)。响应须在毫秒量级时,有界修复能恢复可行;允许秒级停机时,不改写已结束占用的热启动全局重调度在完工时间上仍占优, 但优势幅度已

表 9. E5 与不改写已结束占用的热启动全局重调度(3 个随机种子均值)。「改动比例」为被改写的占用占全表之比。 $C_{\max}$ 一列 R0+ 全区间更优。

算例	预算/s	R0+ $C_{\max}$	R2 $C_{\max}$	改动比例 R0+ / R2
congested_8x4x4	0.2	279.3	299.0	0.61 / 0.57
	1.0	276.0	299.0	0.61 / 0.57
	2.0	275.7	299.0	0.61 / 0.57
example_3x3x2	0.2	78.3	88.7	0.60 / 0.60
	1.0	78.3	88.7	0.60 / 0.60
	2.0	78.3	88.7	0.60 / 0.60

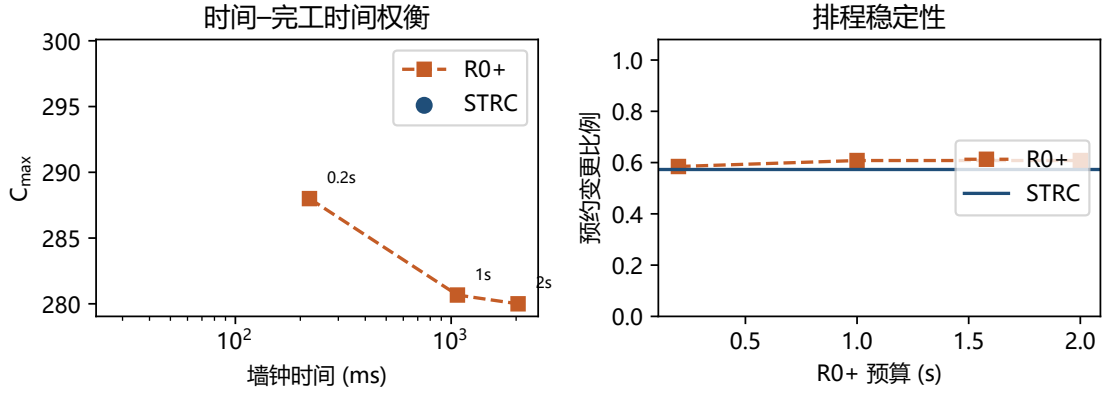


Fig. 7. E5:不改写已结束占用的 R0+ 预算扫描与有界修复定点(congested_8x4x4, 3 个随机种子均值)。左:算法耗时- $C_{\max}$,两条曲线不相交;右:占用改动比例,两法同在六成附近。

远小于从 $t = 0$ 重排的那一档。 $C_{\max}$ 是本领域的主指标,有界修复在其上不优于 R0+,故按它报告。第 5.3 节所说的对照,指的是两者在算法耗时约束下各自可用,不是指存在某个预算区间使 R2 的 $C_{\max}$ 更优。

本表的 R0+ 不改写已经结束的占用:解码把 $t_{\text{end}} \leq t_{\text{now}}$ 的占用预提交,只从决策时刻接着排未开始的工序,并允许改染色体。已结束占用不可改写的审计上,小例与拥堵例各 9 个预算点越界均为 0; 同工件后道已出车、前道还要重定时,改为强制原车、只重规划 t_{now} 之后的尾段,历史空载前缀得以保留。有界修复在同样 6 个算例-随机种子格上仍一律为零。与「从 $t = 0$ 重排」的旧对照相比, $C_{\max}$ 差距大幅收窄,改动比例不再是百分之百对六成;但 18/18 仍无交叉。缺口里原先混着的「改写已经结束的占用」已经被拿掉,剩下的是全局改派改序相对单遍改路的那一截。

6.9 E7:毫秒档内的对照

上一节的对照是热启动种群搜索,预算 0.2–2 s。只用它作对照会留下一个问题:「低两至三个数量级」这个读数,有多少来自预约影响集,又有多少只是因为对照耗时更长? 本节把阶梯中间那两档耗时更短的全局重算补上。

- **RS(全局右移)**。不改路径、不改指派、不改扫描序,把未完成占用整体后推一个延迟 Δ , Δ 取到刚好让受阻走廊上所有可动的占用落到阻断窗之后。不改写已经结束的占用($t_{\text{end}} \leq t_{\text{now}}$ 不可动),与 R2 相同; 因为是统一

表 10. E7 毫秒档内对照(50 对,协议与 E3 相同:繁忙走廊单窗阻断、 $t_{\text{now}} = 0.35 C_{\text{max}}^0$ 、R2 不扩大范围)。四法均 50/50 可行。「已结束占用越界」为改写了已结束占用的格数;「允许改写」为该方法纳入改写的占用占 t_{now} 之后尚未结束占用的比例;未列为该方法对 R2 的 C_{max} 胜负。RA 与 R2 只差允许改写哪些占用,故这两行之差可全部归因于预约影响集。本表不可与表 9 并读——那批只有两个算例。

方法	已结束占用越界	ms(中位)	平均退化	改动比例	允许改写	对 R2 赢/平/输
RS 全局右移	0/50	1.42	58.8%	0.627	—	7/16/27
RD 重解码	0/50	4.16	49.0%	0.618	—	22/14/14
RA 不按影响集划界	0/50	2.76	49.9%	0.604	1.00	0/46/4
R2 有界修复	0/50	2.74	49.8%	0.540	0.92	—

平移,一切间隔要么不变、要么变大,而全部约束都是「 $\geq$ 」型,无需重新路由即可行解。它是「不按影响集划界」的一端:连改路都放弃(第 2.1 节)。

- **RD(原染色体重解码)**。保持机器指派与扫描序,在装了阻断的路由层上走与 R0+ 同一套固定前缀解码。它是本文实现里耗时更短的一次全局重算——种群搜索每评估一个候选都至少要付这笔钱。它不改染色体,因而比 R0+ 耗时更短,也更少改派改序的自由度。
- **RA(不按影响集限制范围,但保留改路)**。把 t_{now} 之后尚未结束的占用全部纳入改写,再走与 R2 完全相同的改路重放。它是「不按影响集划界」的另一端:与 RS 不同,它保留了改路,因而与 R2 只差允许改写哪些占用。两者同一套改路程序,都不改已经结束的占用,共用同一段阻断。因此它是本文唯一能把预约影响集单独隔离出来的对照:两法之差不含改路程序、协议或实现的贡献。它同样不改写已经结束的占用,且已是最大允许改写集合,无需失败后扩大范围。

设置 RA 是为了回答:预约影响集既然已经覆盖尚未结束占用的约 92%(注 2),那么按它划界还剩多少用? E3 比较的是两种不同的划界对象(工序依赖图 vs 预约影响集),而 RA 问的是:要不要按影响集限制范围。

表 10 给出四条读数。

其一,在给出可行方案的各法中,唯一比有界修复更快的是全局右移,而它在完工时间与改写比例上都较差。(按工序依赖图划界的 R1 更快,但它在 0/50 上恢复可行,不构成候选。)RS 只要 1.42 ms,约为 R2 的一半;代价是完工时间平均退化 58.8%(R2 为 49.8%),逐格对比 27/16/7(R2 赢/平/输),且改动的占用反而更多(0.627 对 0.540)——统一平移把每一条尚未结束占用的时刻都改了。若想把右移做得有选择——只推迟真正受影响的车,其余不动——就必须先知道哪些占用受影响,那正是本文划定的范围。右移之所以能不按影响集划界,恰恰因为它放弃了选择。

其二,耗时更短的一次全局重算仍比一次有界修复更长。RD 的耗时中位数是 4.16 ms,为 R2 的 1.5 倍。这条读数与搜索配置无关:任何基于解码的搜索,每评估一个候选都要付至少一次解码的代价。因此上一节「低两至三个数量级」并不是对照选得耗时更长造成的——把对照换成同一实现里耗时更短的那一档,有界修复仍然更快。

其三,不改写已结束占用之后,RD 的完工时间仍略优,但改动更多。全表看 R2 对它是 14/14/22(赢/平/输),平均退化 49.8% 对 49.0%。已结束占用越界已是 0/50:降级后道改为强制原车、只重规划决策时刻之后的尾段,历史空载前缀不再丢。故「RD 更好」不能写成「来自改写已经结束的占用」——这是同染色体、只排未来时,全局改路相对单遍改路的那一截。改动比例 RD 为 0.618,高于 R2 的 0.540:少改写仍在按影响集划界的一侧。

其四,少改写只能从 R2 对 RA 读出,而且读到的只是稳定性。RA 与 R2 只差允许改写哪些占用,因此这一对读数直接回答了「按影响集划界值不值」。预约影响集平均只比平凡上界少纳入 $1 - 0.92$ 的尚未结束占用(逐格 0.80–1.00,取到 1.00 的 4 个格上影响集就是全部尚未结束占用,两法恒等)。就是这不足一成的差别,把改动比例从 0.604 降到 0.540:平均降 0.064、中位降 0.029,逐格 31/18/1(R2 更稳/持平/更差),Wilcoxon 符号秩双侧 $p = 8.8 \times 10^{-7}$ 。

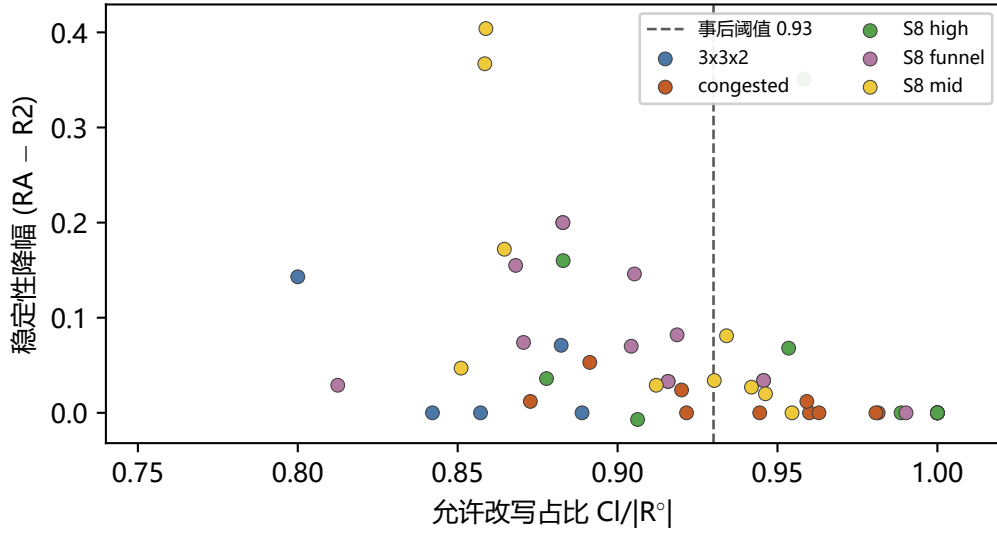


Fig. 8. E7 事后规则:允许改写占比与稳定性降幅(50 对, R2 对 RA)。虚线为样本内阈值 0.93;纵轴为正表示按影响集划界后改动比例更低。

方向高度一致,幅度不大——唯一的反例格降幅为 +0.007,一并记下。与此同时算法耗时几乎相同(2.74 对 2.76 ms),完工时间也几乎相同(比值均值 0.999,逐格 R2 赢/平/输 4/46/0、从未更差,最占优的一格为 0.970)。

这三项合起来才是重点。耗时相同,说明毫秒级响应来自单遍改路、不重开搜索,而不是来自预约影响集(表 1)。完工时间相同,说明影响集不是一种优化手段。改动比例显著更低,说明划界落实的恰是命题 3 承诺的那个量:集外占用的走廊、时段与车辆不变。影响集的作用不在快、也不在缩短 $C_{\max}$,而在标出哪些占用不必改——这解释了为什么幅度与影响集排除掉多少成正比:在影响集接近全部尚未结束占用的算例上(congested_8x4x4,允许改写占比 0.94)降幅只有 0.010,而在影响集排除得多的算例上(S8x4x4_mid,0.91)降幅达 0.118。

把这一正比写成一条事后规则。图 8 以允许改写占比 $CI/|R^o|$ 为横轴、RA 相对 R2 的改动比例降幅为纵轴,画出 50 对。在本样本上取 $CI/|R^o| \geq 0.93$:该线以上 23 格平均降幅 0.029,以下 27 格平均降幅 0.094;占比再升到 0.97 的 8 格降幅为 0(影响集已是平凡上界,两法恒等)。这是一条事后分档,阈值在本表上选定,不作外推定理;它回答的是「何时按影响集划界还值」,不是影响集规模的结构预测(后者仍未建立,第 6.7 节)。

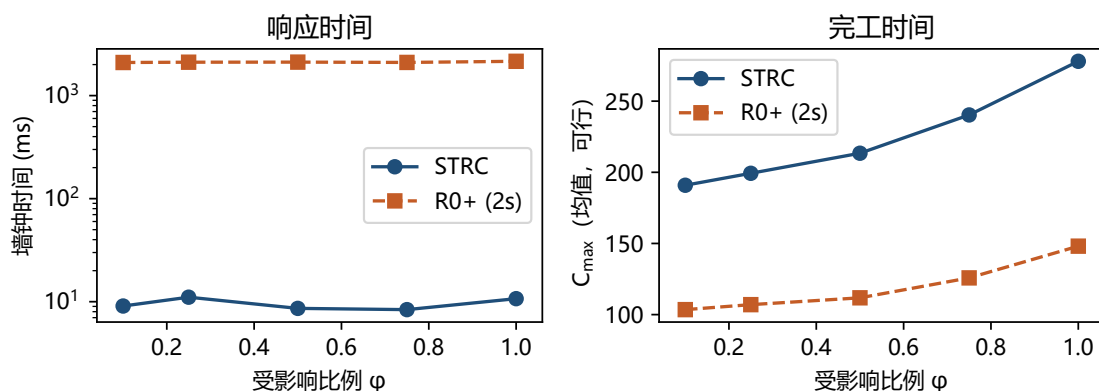
四法并读的结论是:在「不改写已经结束的占用、且响应在毫秒量级」这两个约束下,唯一比有界修复更快的做法(全局右移)在完工时间与改写比例上都更差;不改写已结束占用的重解码在完工时间上仍略优,但更慢、改动也更多;而在同等速度与同等完工时间下,不按影响集划界要多改约 6 个百分点的占用(相对多约一成二)。放宽算法耗时约束,更短的完工时间仍能从热启动搜索得到(第 6.8 节),但那已不是毫秒档。

6.10 扰动规模扫描

表 11 与图 9 报告 φ 扫描(congested_8x4x4, 10 个随机种子; R0+ 预算 2 s; 有界修复含失败后扩大范围)。两法可行率均为 100%; 有界修复耗时约 8–11 ms, R0+ 约 2 s; $C_{\max}$ 仍全由 R0+ 更优,与第 6.8 节一致。小例 example_3x3x2 上同样每格可行(10 个随机种子 $\times$ 五个 φ 值),有界修复约 1 ms 量级,加速比 87 $\times$ –3051 $\times$ (跨两个算例的逐格极值)。

表 11. 扰动规模对照(congested_8x4x4, 10 个随机种子平均)。有界修复含扩大范围;R0+ 预算 2s。

φ	STRC 可行	STRC ms	R0+ $C_{\max}$	耗时比
0.10	100%	9.1	103.5	406×
0.25	100%	11.1	107.0	325×
0.50	100%	8.6	111.8	333×
0.75	100%	8.4	125.8	254×
1.00	100%	10.7	148.1	203×

Fig. 9. 扰动规模扫描(congested_8x4x4, 10 个随机种子): 算法耗时(对数轴)与完工时间随 φ 变化。

这里有界修复打开了失败后扩大范围,故与 E3/E5 不可直接比较:扩大范围是把可行率拉到 100% 的步骤,本身已有成例(第 2.1 节)。本表的用途是说明,即使在扰动比例升至 $\varphi = 1$ (全部未来工序受扰)时,按影响集划界的路径仍停留在十余毫秒量级,算法耗时不随扰动规模崩溃。

6.11 E8:算例规模上的外推

上一节扫的是扰动的规模,算例始终是 8 工件 4 机。本节换一维:固定扰动协议,把算例本身放大。这一问的要害不在耗时——耗时不崩是可以预期的——而在影响集占比会不会随规模涨到接近 1,使「有界」名存实亡。

梯子取工件数 $2k$ 、机器数 k 、车数 $k, k = 4, \dots, 10$, 即 $8 \times 4 \times 4$ 到 $20 \times 10 \times 10$, 共 7 档 $\times 10$ 个随机种子 = 70 对。拥堵档、异构度、柔性度、运输/加工时长比与装卸点最小割、远端割全部按同一目标标定,只有规模变。三条对照与第 6.9 节同定义。

影响集占比不随规模增长。 从 0.56 降到 0.48,趋势甚至是缓降的。这是本节最要紧的一条:有界性并未随规模被稀释。但须说明这不是一个渐近主张——七档不足以拟合任何增长率,能说的只是在这段区间内它没有退化。

算法耗时随占用总数近似线性。 $|R|$ 增至 2.7 倍时, R2 的耗时中位数从 2.9 增至 10.4 ms,仍在十毫秒量级。第 1 级改路的可行格数为 69/70; 唯一失败的一格($10 \times 5 \times 5$, 随机种子 42)由失败后扩大范围在第 3 轮兜住,总耗时 12.0 ms、完工时间与首级尝试相同,故终可行为 70/70。

表 12. E8 算例规模扫描(7 档 $\times$ 10 个随机种子)。「影响集」为预约影响集占全表之比;耗时取中位数。末列为 R2 对 RS 的 $C_{\max}$ 胜负。

规模	R	影响集	R2 ms	RD ms	RD/R2	R2 对 RS
$8 \times 4 \times 4$	148	0.561	2.9	4.4	1.5 $\times$	7/1/2
$10 \times 5 \times 5$	172	0.485	3.2	6.1	1.9 $\times$	9/0/1
$12 \times 6 \times 6$	230	0.481	4.5	10.7	2.4 $\times$	7/1/2
$14 \times 7 \times 7$	281	0.492	6.1	14.5	2.4 $\times$	8/0/2
$16 \times 8 \times 8$	291	0.534	6.4	16.4	2.6 $\times$	10/0/0
$18 \times 9 \times 9$	356	0.535	8.2	26.4	3.2 $\times$	10/0/0
$20 \times 10 \times 10$	402	0.476	10.4	34.2	3.3 $\times$	10/0/0

两个对照的差距随规模拉大。耗时更短的全局重算与有界修复的耗时比从 1.5 $\times$ 升到 3.3 $\times$ ——规模越大,重算一遍越不划算;而对不按影响集划界的右移,R2 的 $C_{\max}$ 胜负从 7/1/2 变为最后三档的 10/0/0——规模越大,「只改须改的占用」相对「整体后推」的优势越明显。

口径限制。这条梯子只有一个拥堵档、一个算例随机种子(每档的算例本身由随机种子 42 生成),因而它检验的是规模这一维,不能替代第 6.1 节主批次在拥堵结构上的对照。第 6.5 节的四类扰动交叉与第 6.10 节的扰动规模扫描也没有在这批算例上重跑。

6.12 外部布局批次的复现

第 6.1 节第二批把布局出处换成外部来源后,50 个算例-随机种子对上 E1/E2/E3/E5 的读数与主批次逐项同向(E4 是唯一被改写的一格,见本节末;E5 与主批次的唯一差别是多出一个两法持平的预算点,见下):工序依赖图给出空范围 50/50(结构泄漏计 0),结构包含性 50/50,集外字段逐条不变 50/50;边界消融下第 1 级改路的可行格数为 0/50,按预约影响集划界为 50/50。E1 协议下的影响集占比逐布局中位落在 0.438–0.547,与主批次逐算例的 0.54–0.58 同量级。与全局重调度的权衡亦无变化:17/18 个预算点上 R0+ 的 $C_{\max}$ 更优,余下 1 点两法持平(Lyul6_8m,随机种子 2024,预算 0.2 s),没有一个点上有界修复更优,交叉点仍不存在,而有界修复的算法耗时仍在毫秒量级、R0+ 仍在秒量级。

这一批的意义不在于给出新的性能数字,而在于回答“这些现象是否只在本文自己设计的布局上成立”:B 类扰动使按工序依赖图划界得到空范围、预约影响集包含之、集外字段保持原计划,这三件事在一批不由本文选定的拓扑上同样成立。唯一被这批算例改写的结论是结构可预测性(第 6.7 节),而它本来就是负结果。

6.13 小结

- E1 与边界消融(E3)在 50 个算例-随机种子上无一例外成立;案例甘特把同一协议落到工序时间轴。
- 划界覆盖矩阵(E6)在四类扰动 $\times$ 50 对上给出划界读数: A/B 二分无例外,且 A 类上预约影响集一致包含工序依赖图划界;修复一侧四类都开口: B 类第 1 级均满(50/50/ 50/50),车辆故障换车 50/50,机械臂故障改派 50/50。
- 结构包含性(E2a)与集外字段逐条不变(E2b)均满通过;但这只说明命题 3 的四条前提在本文协议下成立,命题仍只作充分条件(注 4 记录了前提 (iii) 曾如何在实现上被破坏)。
- 与全局重调度相比,有界修复的 $C_{\max}$ 全区间较差、无交叉;算法耗时为毫秒对秒,改动比例已与不改写已结束占用的 R0+ 同在 50%–66% 量级。
- 毫秒档内对照(E7)排除了「对照耗时更长」这一疑问:在给出可行方案的各法中,唯一比有界修复更快的是不按影响集划界的全局右移,而它在 $C_{\max}$ (逐格 27/16/7)与改写比例 (0.627 对 0.540)上都较差;而同一实现里耗时更短的一次全局重算(原染色体重解码)比一次有界修复还长 1.5 倍;RD 已走固定前缀,已结束占用越界 0/50。

- 同一批还把预约影响集单独隔离出来测了一次:换用「不按影响集限制范围、把尚未结束占用全部纳入改写」的对照(RA, 与 R2 同一套改路、都不改已经结束的占用),算法耗时与 $C_{\max}$ 几乎不变(逐格 R2 赢/平/输 4/46/0), 改动比例却由 0.540 升到 0.604——逐格 31/18/1, $p = 8.8 \times 10^{-7}$ 。故毫秒级响应来自单遍改路而非影响集;少改写只能从这一对照读出, 且降幅与影响集排除掉多少成正比;样本内事后规则是允许改写占比超过 0.93 时平均降幅从 0.094 落到 0.029(图 8)。
- 算例规模扫描(E8)把梯子推到 20 工件 10 机:影响集占比从 0.56 缓降到 0.48(有界性未随规模稀释),算法耗时随占用总数近似线性升至 10.4 ms,而与两个对照的差距同向拉大。这是一段区间内的读数,不是渐近主张。
- 预约影响集覆盖约 92% 的尚未结束占用,不作最小性声称。让行边纳入接壤后,差分试探在 50 对上泄漏为 0(0/50); 这只关闭了已观测到的那条通道。
- 三档非均匀边权上,工序依赖图给出空范围与 R1 全败在 150 格上复现;第 1 级 R2 可行率在对数正态与距离梯度为 50/50/50/50,二分边权为 49/50。空范围对边权不敏感;可行率只在二分边上掉了一格。
- 结构可预测性(E4)是负结果,且换用外部来源布局后更清楚:在同一张 4×4 外部路网上只改机器落位,影响集占比即变动 0.207(已是三张不同自建路网之间极差 0.051 的约四倍),而 LU 最小割与漏斗占比在整批外部算例上取值恒定,无从解释这一变动。替代方向指向需求侧的时序密度。
- 两批算例承担不同的问题:主批次给出现象与机制,第二批只检验这些现象是否依赖本文自选的布局。把布局出处换成外部来源后(50 对),E1/E2/E3/E5 的读数逐项复现,无一条结论因换布局而改变(第 6.12 节)。第二批用来收窄结构可预测性的结论,且只有 E4 一格必须两批合读(第 6.1 节)。

7 总结

本文指认了一类工序依赖图覆盖不到的损坏,并把重调度范围改到走廊占用及其让行、接续关系上,得到由这些关系唯一确定的预约影响集(STRC)。在 5 算例 $\times$ 10 个随机种子共 50 对上:走廊阻断时按工序依赖图划界 50/50 为空,而原排程确已不可行;同一套改路程序下只替换划界对象时,按工序依赖图划界 0/50 恢复可行,按预约影响集划界 50/50;影响集的结构闭合性 50/50 成立,集外占用的走廊、时段与车辆逐条不变 50/50 成立;四类扰动 $\times$ 两种划界的覆盖矩阵上 A/B 二分无例外,且 A 类上预约影响集一致包含工序依赖图划出的范围。

让行关系、集内改路而集外保持原计划、以及失败后扩大允许改写的范围,分别已见于铁路 alternative graph、无冲突多车路径与 Local Rescheduling (第 2.5 节)。这些步骤用来落实影响集。本文主张的是:工序依赖图按定义不含走廊占用之间的让行关系,因而覆盖不到只使占用失效的扰动;改到预约依赖图上之后,须改写的占用由既定先后唯一确定,而不是一个启发式半径。

把允许改写的集合从预约影响集换成「 t_{now} 之后尚未结束的全部占用」,而保持同一套改路程序、都不改已经结束的占用、协议也不变时,算法耗时与完工时间几乎不动,改动比例却由 0.540 升到 0.604(逐格 31/18/1, $p = 8.8 \times 10^{-7}$; 第 6.9 节)。故毫秒级响应来自单遍改路、不重开搜索;少改写只能从这一对照读出,落实的是命题 3 承诺的量:集外占用保持原计划。收益幅度与影响集排除掉多少成正比,因而在影响集接近平凡上界时趋于消失。

7.1 局限

以下四条按对前文读数的影响排序,第 7.2 节逐条给出推进方向。

- (1) 完工时间上全区间劣于不改写已结束占用的热启动全局重调度,且不存在交叉点。在 18 个预算点上无一例外。对照已走固定前缀解码,已结束占用不可改写的审计为 0(第 6.8 节);E7 上 RD 越界同样为 0/50。差距比从 $t = 0$ 重排时小得多,但方向没变:有界修复换来的是算法耗时,交出的是完工时间。第 6.9 节已排除「对照耗时更长」:同一实现里耗时更短的一次全局重算比有界修复更慢,且已走固定前缀;少改写只能从与 RA 的对照读出,不能从对 R0+ 读出。

- (2) 对预约影响集,保证仍只是充分条件,边集未经独立穷尽检验,允许改写的集合不紧,规模也不可由只看路网的指标预测。 四者分述。
- (i) 命题 3 只是充分条件,50/50 不等于充要: 实验说明的是四条前提在本文协议下成立,不是它们可以省略。前提 (iii) 尤其脆弱—— 只要实现以比占用更粗的粒度决定改路范围, U 外的占用就会被顺带改写 (注 4); 本文没有给出一个能自动检出该类粒度错配的手段。逐条比对集外字段(E2b)不足以兜住这类错配: 同一注记的后半段记录了一次只发生在影响集内部、因而 E2b 查不到的、对已结束占用的改写,它要靠一项独立的审计才暴露出来。因此本文对该类缺陷的检出依赖于「事后想到该查什么」,而非一个完备的检查。
- (ii) 依赖边集并不穷尽物理耦合,但已观测到的那条接壤通道已经关掉。 E2b 满通过检验的是修复实现与允许改写集合是否一致,不是四类边覆盖了一切传播通道。差分试探—— 只把影响集内一条占用后移一个单位,观察集外是否出现新的同走廊重叠—— 在把接壤并入让行边并重跑主矩阵之后, 50 对上泄漏为 0(0/50)。同车方向始终为 0。这不是完备性证明: 试探只覆盖「延后 1 个时间单位」这一种扰动,其它耦合通道仍未检验(注 2)。
- (iii) 预约影响集不作最小性声称,且规模并不小: 它覆盖 t_{now} 之后尚未结束占用的约 92%(注 2); 相对「把尚未结束的占用全部纳入改写」这一平凡上界, 少纳入的不足一成。这一条的幅度已实测(第 6.9 节的 RA): 那不足一成的差别把改动比例降了 0.064(逐格 31/18/1, $p = 8.8 \times 10^{-7}$), 而算法耗时与 C_{max} 不变。限制不在于「划界无用」,而在于收益幅度不大且与影响集排除掉多少成正比—— 在影响集接近全部尚未结束占用的算例上降幅仅 0.010。事后规则(图 8)把「何时」落成一条样本内分档: $CI/|R^o| \geq 0.93$ 时平均降幅 0.029(23 格), 低于该线为 0.094(27 格); 它不是结构预测,也不能外推。
- (iv) 影响集规模的结构可预测性未能建立,且已知不由静态割给出: 装卸点最小割与漏斗占比都没有做出可复现的区分; 换用外部来源布局后这一点变得更确定—— 这两个指标在整批外部算例上取值恒定(两张路网都把装卸站放在网格角点,四邻接下角点恰有两条出边),而影响集占比在同一张路网上仅换机器落位就变动 0.207(第 6.7 节)。这条否定只针对「静态割能解释影响集规模」,不是「路网几何与影响集规模无关」—— 后者需要更多张互不相同的公开路网才能检验,而可得的路网就这么多。下一个候选刻画应是 t_{now} 邻域内走廊占用的时序密度,本文未做。
- (3) 实现覆盖仍窄于正文所述: 修复一侧已满、恢复动作只有一档半、只做一次响应。 扰动一维,四类都有注入通道且修复均满 (50/50/ 50/50/ 50/50/ 50/50,第 6.5 节)。 动作一维,B 类仍只改路; A 类开口换车与改派,仍不改扫描序。放开改序即向全局重调度退化,「有界」也随之失去边界。 时间一维,由假设 A3,扰动在 t_{now} 完全已知且不叠加; 扰动流、预测误差与滚动响应均未涉及。 规模一维的情况好一些但仍有限: 第 6.11 节把梯子推到 20 工件 10 机、70 对,影响集占比未随规模上升; 但那条梯子只有一个拥堵档与一个算例随机种子,7 档也不足以支撑任何渐近论断,更大规模与更多结构下的行为仍未知。
- (4) 外部参照缺位: 考虑执行期扰动的这一类研究没有可换的公开基准,算例出处只外部化了一半, 读数亦未在第二套实现上复现。 带无冲突运输的柔性作业车间已有若干公开算例族,但执行期扰动这一类没有: 公开族发布的都是静态问题的输入。更硬的一层障碍在算例格式—— 主流族把运输时间发布为机器间 行程时间矩阵而不是路网,我们对两族共 11 张布局逐张检验过 [3, 6],没有一张能还原成与所发布矩阵一致的无向走廊图(全部不对称,即同一对位置往返时长不等; 其中两张连有向图也还原不出,因为违反三角不等式)。矩阵里没有走廊,就没有走廊占用,于是 R 、 G_R 、 $CI(\cdot)$ 与 B 类扰动在这类算例上一概无从定义。这比静态工作受到的约束更紧: 静态工作可以把争用约束退化掉,退化后求解的仍是文献求解的那个问题,因而仍能借公开算例标定搜索能力; 本文没有这条退路—— 退化掉争用,被研究的对象本身就不存在了。唯一发布显式双向排他路网的是 Lyu 等 [10]。本文已按其附录布局做出第二批算例(第 6.1 节),布局因此不再由本文设计; 但外部化只做到拓扑一层—— 逐段行驶时间与工件数据仍是本文的。前者原文未随该批布局发表(只随单个示例算例

给出,且取值并不均匀),后者若一并借用,公开族过低的运输/加工时长比会使争用现象直接消失。所以这批算例不能用来复现 Lyu 或 van Os [20] 的参照值;后者的模型另有「一个节点同时只容一车」的约束,与本文的走廊排他语义也不是同一回事。算例出处这一条因此只解决了一半:布局那一半解决了,时间标定那一半没有。等权之外另扫了三档非均匀边权(对数正态、离装卸点越远越慢、随机 0.5/2),只重跑 E1/E3: 工序依赖图给出空范围与 R1 全败在 150 格上复现;R2 第 1 级可行率分别为 50/50、50/50、49/50。空范围对边权不敏感;第 1 级可行率只在二分边上掉了一格(49/50),距离梯度在补上接壤边后回到 50/50。这不能替代一份同时发布路网与逐段时长的外部算例族。外部批次的覆盖范围也窄于主批次:它复现的是 E1/E2/E3/E5(第 6.12 节),E6 的四类扰动交叉、扰动规模扫描以及第 6.9 与 6.11 节的两批(毫秒档内对照、算例规模)都没有在外部布局上重跑;这四项的读数目前只有自建算例支撑。此外,该批的 5 个算例只落在 2 张路网上(第 6.1 节),路网几何这一维的样本量因此很小。最后,下层路网、时间窗路由与校验器与静态一体化工作复用同一实现:这保证了归因的干净,但也意味着上述读数尚未在第二套独立实现上复现。

7.2 后续工作

下列四条与上一节的四个局限一一对应。

一、正视完工时间上的全区间劣势(局限 1)。前缀解码对已结束占用的改写已在主批次收干净(E5 的 18 个预算点越界 0, E7 上 RD 为 0/50)。局限 1 剩下的就是 $C_{\max}$ 本身:不打算换加权指标绕过去。若要缩小差距,需要的是在不改写已经结束占用的前提下做有限度改派或改序,那会向全局重调度退化。

二、把预约影响集的保证做实、做紧,并给它的规模找一个刻画(局限 2)。做实:接壤已并入让行边,主矩阵重跑后 E1/E2/E3 仍满通过,试探泄漏为 0。下一步是换一种扰动形态(例如缩短占用、或延后超过 1 个单位)再探,而不是再补同一条边。做紧,指向最小允许改写集合:在固定修复能力下定义最小性,并给出影响集与之的差距,这是把「有界」从充分条件推进到紧界的必经一步;当前影响集覆盖约 92% 的尚未结束占用,差距很可能不小。刻画规模已有一条事后规则(允许改写占比,图 8),但它随排程而变;要换一族可在扰动前计算的指标:第 6.7 节已把静态割这一族排除——它在真实布局上取值恒定,而同一张路网上仅改机器落位,影响集占比就变动 0.207。后者正好提示了方向:起作用的似乎是需求落在哪里而不是路网提供了什么。下一个候选是 t_{now} 邻域内的走廊占用时序密度,例如受扰走廊在其繁忙窗口内的重叠占用条数、以及该走廊在让行图上的出度。这类量随排程而变,不再是纯拓扑量,因而也需要重新界定「可预测性」问的是:是给定布局预测影响集规模,还是给定布局与当前排程预测它。

三、放开假设 A3(局限 3;A3 为单次、完全观测)。覆盖矩阵修复一侧四类均已满:阻断与走廊通行变慢第 1 级、车辆故障换车、机械臂故障改派。下一步不是再开改序,而是放开该条,考察影响集在连续扰动下是否会累积膨胀,以及两次修复之间是否需要显式的「归位」步骤。

四、把时间标定也外部化(局限 4)。布局出处已解决一半;等权之外的三档边权说明工序依赖图给出空范围这一命题站住了,第 1 级可行率没有。要补齐另一半,仍然需要一份同时发布排他路网与逐段时长的算例族——自建边权扫不能替代它。至于第二套独立实现,它不必单独立项——上述任何一件由他人复现时都会顺带完成。

References

- [1] Riyad J. Abumaizar and Joseph A. Svestka. 1997. Rescheduling job shops under random disruptions. *International Journal of Production Research* 35, 7 (1997), 2065–2082. doi:10.1080/002075497195074
- [2] M. Selim Aktürk and Elif Görgülü. 1999. Match-up scheduling under a machine breakdown. *European Journal of Operational Research* 112, 1 (1999), 81–97. doi:10.1016/S0377-2217(97)00396-2
- [3] Ümit Bilge and Gündüz Ulusoy. 1995. A Time Window Approach to Simultaneous Scheduling of Machines and Material Handling System in an FMS. *Operations Research* 43, 6 (1995), 1058–1070. doi:10.1287/opre.43.6.1058
- [4] Andrea D'Ariano, Dario Pacciarelli, and Marco Pranzo. 2007. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research* 183, 2 (2007), 643–657. doi:10.1016/j.ejor.2006.10.034

- [5] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production–Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.
- [6] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6
- [7] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139
- [8] Jürgen Kuster, Dietmar Jannach, and Gerhard Friedrich. 2007. Local Rescheduling: A novel approach for efficient response to schedule disruptions. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched)*. IEEE, 79–86. doi:10.1109/SCIS.2007.367673
- [9] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543
- [10] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109
- [11] Nir Malka, Guy Shani, and Roni Stern. 2024. Online Planning for Multi Agent Path Finding in Inaccurate Maps. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 10214–10221. doi:10.1109/IROS58592.2024.10801975
- [12] Alessandro Mascis and Dario Pacciarelli. 2002. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research* 143, 3 (2002), 498–517. doi:10.1016/S0377-2217(01)00338-1
- [13] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355
- [14] Djamila Ouelhadj and Sanja Petrovic. 2009. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* 12, 4 (2009), 417–431. doi:10.1007/s10951-008-0090-8
- [15] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352
- [16] Velusamy Subramaniam and Amritpal Singh Raheja. 2003. mAOR: A heuristic-based reactive repair mechanism for job shop schedules. *The International Journal of Advanced Manufacturing Technology* 22, 9–10 (2003), 669–680. doi:10.1007/s00170-003-1601-6
- [17] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526
- [18] Jiri Svancara, Marek Vlk, Roni Stern, Dor Atzmon, and Roman Barták. 2019. Online Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 7732–7739. doi:10.1609/aaai.v33i01.33017732
- [19] Hongtao Tang, Xuwei Zhang, Jiawei Huang, Xuesong Xu, and Jiansha Lu. 2026. Integrated Scheduling of Multi-Objective Lot-Streaming Hybrid Flowshop with AGV under Dynamic Environments. *International Journal of Industrial Engineering Computations* 17, 2 (2026), 295–316. doi:10.5267/j.ijiec.2025.9.003
- [20] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342
- [21] Guilherme E Vieira, Jeffrey W Herrmann, and Edward Lin. 2003. Rescheduling Manufacturing Systems: A Framework of Strategies, Policies, and Methods. *Journal of Scheduling* 6, 1 (2003), 39–62. doi:10.1023/A:1022235519958
- [22] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795
- [23] Yannik Zeiträg and José Rui Figueira. 2025. A novel machine-learning rolling horizon heuristic for dynamic lot-sizing and job shop scheduling problems. *International Journal of Production Research* 63, 12 (2025), 4563–4589.
- [24] Hongliang Zhang, Chaoqun Qin, Wenhui Zhang, Zhenxing Xu, Gongjie Xu, and Zhenhua Gao. 2023. Energy-Saving Scheduling for Flexible Job Shop Problem with AGV Transportation Considering Emergencies. *Systems* 11, 2 (2023), 103. doi:10.3390/systems11020103
- [25] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*.
- [26] Ziyang Zhao, Shixin Liu, MengChu Zhou, Xiwang Guo, and Liang Qi. 2019. Decomposition method for new single-machine scheduling problems from steel production systems. *IEEE Transactions on Automation Science and Engineering* 17, 3 (2019), 1376–1387.